

copies of the object, and are managed by different servers. When a process looks up a name, it is given the entire set of capabilities. To see how this feature might be of use, consider the library procedure *open* for opening a file. It looks up a file and gets a capability set in return. It then tries each of the capabilities in turn until it finds one whose server is alive. In this way, if one object is unavailable, another one can be used in its place. It should be clear that this mechanism works best when the files are immutable, so there is no danger that any of them will have changed since they were created.

Third, a row may contain multiple columns, each forming a different protection domain and having different rights. For example, a directory may have one column for the owner, one for the owner's group, and one for everyone else, to simulate the UNIX protection scheme. A capability for a directory is really a capability for a specific column in a directory, making it possible for the owner, group, and others to have different permissions. Since the underlying capability set is the same for all columns of a row, it is only necessary to store the rights bits for each column. The actual capabilities can be computed as needed.

The layout of an example directory with five entries is shown in Fig. 7-22. This directory has one row for each of the five file names stored in it. The directory has three columns, each one representing a different protection domain, in this case for the owner, the owner's group, and everyone else. When the directory owner gives away a capability for, say, the last column, the recipient has no access to the more powerful capabilities in the first two columns.

As we mentioned above, directories may contain capabilities for other directories. This ability allows us to build not only trees but also directory graphs in their full generality. One obvious use of this power is to place the capability for a file in two or more directories, thus creating multiple links to it. These capabilities may also have different rights, making it possible for people sharing a file to have different access permissions, something impossible in UNIX.

In any distributed system, especially one intended for use on wide-area networks, it is difficult to have any concept of a single, global root directory. In Amoeba, every user has his own root directory, as shown in Fig. 7-23. It contains capabilities for not only the user's private subdirectories, but also for various public directories containing system programs and other shared files.

Some of the directories in each user's root are similar to those in UNIX, such as *bin*, *dev*, and *etc*. However, others are fundamentally different. One of these is *home*, which is the user's home directory.

Another is *public*, which contains the start of the shared public tree. Here we find *cap*, *hosts*, and *pool*, among others. When a process wants to contact the bullet server, the directory server, or any other server, for example, to create a new object, it must have a generic capability for talking to that server. These capabilities are kept in */public/cap*.

Another directory in *public* is *hosts*, which contains a directory for each

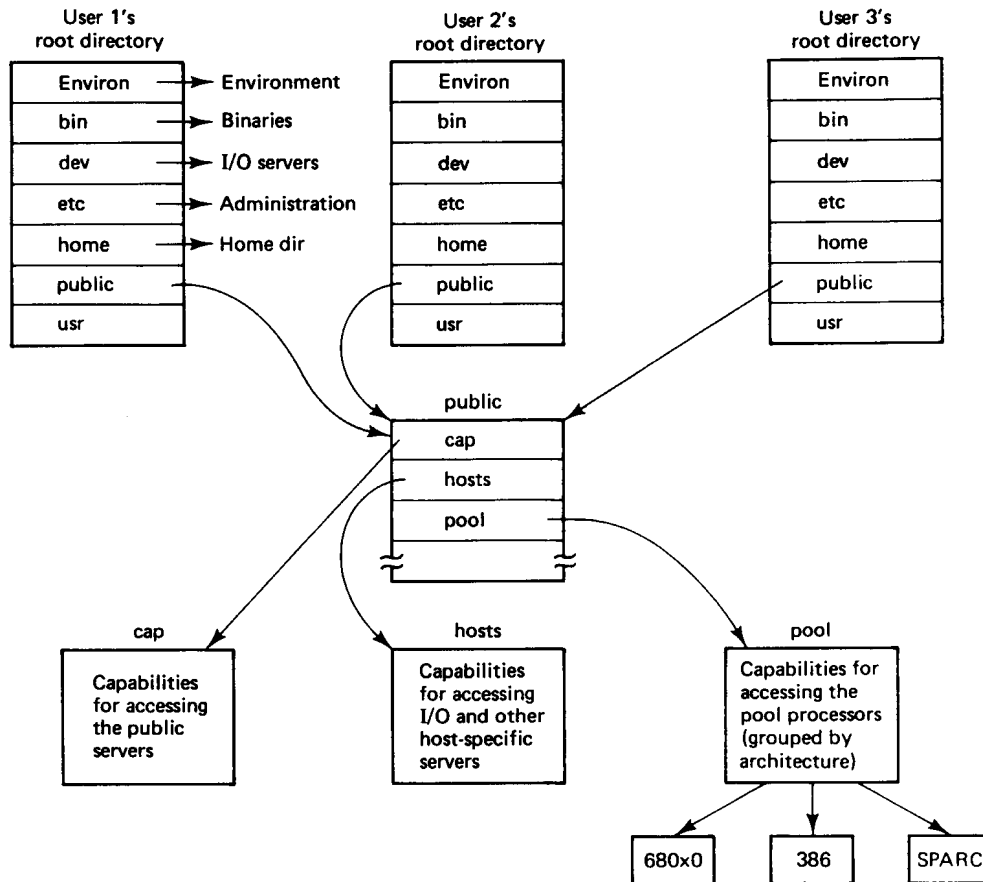


Fig. 7-23. A simplified version of the Amoeba directory hierarchy.

machine in the system. This directory contains capabilities for various servers that can be found on a host, such as a disk server, a terminal server, a process server, a random number server, and so on. Finally, *pool* contains capabilities for the pool processors, grouped by CPU architecture. A mechanism is present to restrict each user to a specific set of pool processors.

The Directory Server Interface

The principal directory server calls are listed in Fig. 7-24. The first two, *create* and *delete*, are used to make and remove directories, respectively. When a directory is created, its capability is returned, just as with making any object. This capability can subsequently be inserted into another directory to build a hierarchy. This low-level interface gives maximum control over the shape of

the naming graph. Since many programs are content to work with conventional directory trees, a library package is available to make this easier.

Call	Description
Create	Create a new directory
Delete	Delete a directory or an entry in a directory
Append	Add a new directory entry to a specified directory
Replace	Replace a single directory entry
Lookup	Return the capability set corresponding to a specified name
Getmasks	Return the rights masks for the specified entry
Chmod	Change the rights bits in an existing directory entry

Fig. 7-24. The principal directory server calls.

It is worth noting that deleting a directory entry is not the same as destroying the object itself. If a capability is removed from a directory, the object itself continues to exist. The capability can be put into another directory, for example. To get rid of the object, it must explicitly be destroyed or garbage collected.

To add a new entry to a directory, be it a file, a directory, or another kind of object, the *append* call is used. Like most of the directory server calls, it specifies the capability of the directory to be used (added to), as well as the capability to put in the directory and the rights bits for all the columns. An existing entry can be overwritten with *replace*, for example, when a file has been edited and the new version is to be used instead of the old one.

The most common directory operation is *lookup*. Its parameters are a capability for a directory (column) and an ASCII string. It returns the corresponding capability set. Opening a file for reading requires first looking up its capabilities.

The last two operations listed are for reading and writing the rights masks for all the columns in a row specified by its string.

A few other directory operations also exist. These are mostly concerned with looking up or replacing multiple files at the same time. They can be useful for implementing atomic transactions involving multiple files.

Implementation of the Directory Server

The directory server is a critical component in the Amoeba system, so it has been implemented in a fault-tolerant way. The basic data structure is an array of capability pairs stored on a raw disk partition. This array does not use the bullet

server because it must be updated frequently and the overhead was thought to be too much.

When a directory is created, the object number put into its capability is an index into this array. When a directory capability is presented, the server inspects the object number contained in it and uses it to fetch the corresponding capability pair from the array. These capabilities are for identical files, stored on different bullet servers, each of which contains the directory and the *Check* field used to verify the authenticity of the directory capability.

When a directory is changed, a new bullet file is created for it, and the arrays on the raw disk partition are overwritten. The second copy is created later by a background thread. The old directories are then destroyed. Although this mechanism has some extra overhead, it provides a much higher degree of reliability than traditional file systems. In addition, normally, directory servers come in pairs, each with its own array of capability pairs (on different disks), to prevent disaster if one of the raw disk partitions is damaged. The two servers communicate to keep synchronized. It is also possible to run with only one. The two-server mode is shown in Fig. 7-25.

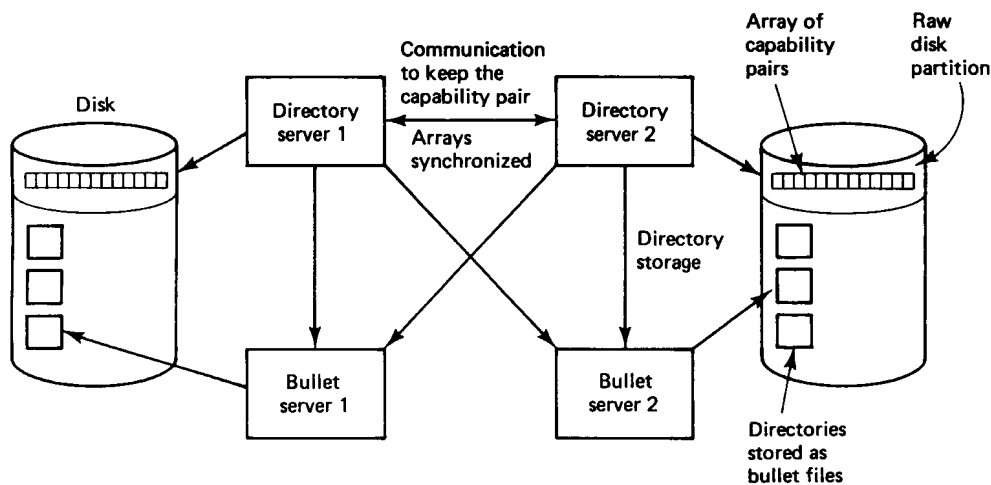


Fig. 7-25. A pair of directory servers. All data are stored twice, on different bullet servers.

In Fig. 7-22 the capability set is shown as being stored only once per row, even though there are multiple columns. This organization is actually used. In most cases, the *owner* column contains rights bits that are all 1s, so the capabilities in the set are true owner capabilities (i.e., the *Check* field has not been run through the one-way function). When a name in another column is looked up, the directory server itself computes the restricted capability by XORing the *rights*

field taken from the directory entry with the *Check* field taken from the owner capability. This result is then run through the one-way function and returned to the caller.

This method eliminates the need to store large numbers of capabilities. Furthermore, the directory server caches heavily used capabilities to avoid unnecessary use of the one-way function. If the capability set does not contain owner capabilities, the server has to be invoked to compute the restricted capabilities because the directory server then does not have access to the original *Check* field.

7.6.3. The Replication Server

Objects managed by the directory server can be replicated automatically by using the replication server. It practices what is called **lazy replication**. What this means is that when a file or other object is created, initially only one copy is made. Then the replication server can be invoked to produce identical replicas, when it has time. Instead of making direct calls to it, the replication server is kept running in the background all the time, scanning specified parts of the directory system periodically. Whenever it finds a directory entry that is supposed to contain n capabilities but contains fewer, it contacts the relevant servers and arranges for additional copies to be made. Although the replication server can be used to replicate any kind of object, it works best for immutable objects, such as bullet files. The advantage is that immutable objects cannot change during the replication process, so it can safely go in the background, even if it takes a substantial time. Mutable objects might change during the replication process, adding additional complexity to avoid inconsistency.

In addition, the replication server runs the aging and garbage collection mechanism used by the bullet server and other servers. Periodically, it touches every object under the directory server's control, to prevent them from timing out. It also sends the *age* messages to the servers to cause them to decrement all the object counters and garbage collect any that have reached zero.

7.6.4. The Run Server

When the user types a command (e.g., *sort*) at the terminal, two decisions must be made:

1. On what architecture type should the process be run?
2. Which processor should be chosen?

The first question relates to whether the process should run on a 386, SPARC, 680x0, and so on. The second relates to the choice of the specific CPU and

depends on the load and memory availability of the candidate processors. The **run server** helps make these decisions.

Each run server manages one or more processor pools. A processor pool is represented by a directory called a **pooldir**, which contains subdirectories for each of the CPU architectures supported. The subdirectories contain capabilities for accessing the process servers on each of the machines in the pool. An example arrangement is shown in Fig. 7-26. Other arrangements are also possible, including mixed and overlapping pools, and dividing pools into subpools.

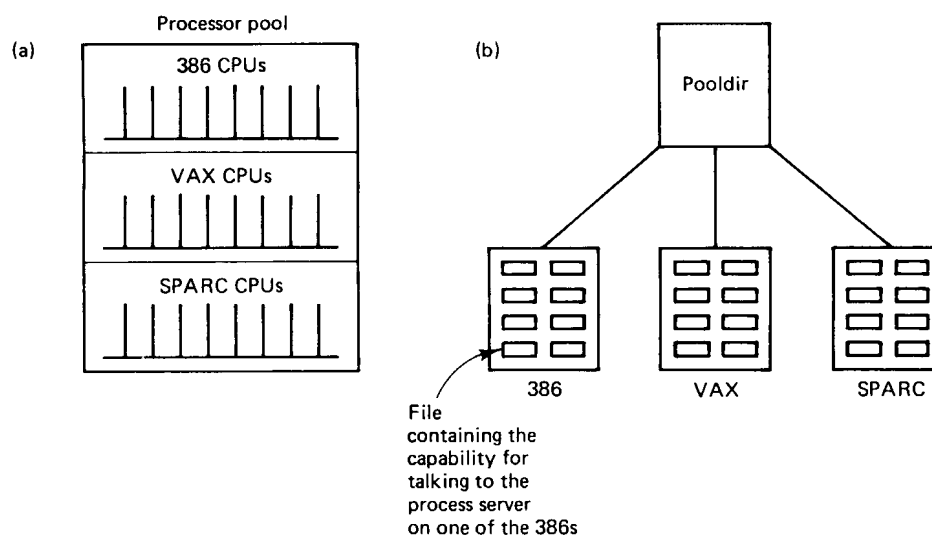


Fig. 7-26. (a) A processor pool. (b) The corresponding pooldir.

When the shell wants to run a program, it looks in */bin* to find, say, *sort*. If *sort* is available for multiple architectures, *sort* will not be a single file, but a directory containing executable programs for each available architecture. The shell then does an RPC with the run server sending it all the available process descriptors and asking it to pick both an architecture and a specific CPU.

The run server then looks in its pooldir to see what it has to offer. The selection is made approximately as follows. First, the intersection of the process descriptors and pool processors is computed. If there are process descriptors (i.e., binary programs) for the 386, SPARC, and 680x0, and this run server manages 386, SPARC, and VAX pool processors, only the 386 is a possibility, so the other machines are eliminated as candidates.

Second, the run server checks to see which of the candidate machines have enough memory to run the program. Those that do not are also eliminated. The run server keeps track of the memory and CPU usage of each of its pool

processors by making *getload* calls to each one regularly to request these values, so the numbers in the run server's tables are continuously refreshed.

Third, and last, for each of the remaining machines, an estimate is obtained of the computing power that can be devoted to the new program. Each CPU makes its own estimate. The heuristic used takes as input the known total computing power of the CPU and the number of currently active threads running on it. For example, if a 20-MIPS machine currently has four active threads, the addition of a fifth one means that each one, including the new one, will get 4 MIPS on the average. If another processor has 10 MIPS and one thread, on this machine the new program can expect 5 MIPS. The run server chooses the processor that can deliver the most MIPS and returns the capability for talking to its process server to the caller. The caller then uses this capability to create the process, as described in Sec. 7.3.

7.6.5. The Boot Server

As another example of an Amoeba server, let us consider the *boot server*. The boot server is used to provide a degree of fault tolerance to Amoeba by checking that all servers that are supposed to be running are in fact running, and taking corrective action when they are not. A server that is interested in surviving crashes can be included in the boot server's configuration file. Each entry tells how often the boot server should poll and how it should poll. As long as the server responds correctly, the boot server takes no further action.

However, if the server should fail to respond after a specified number of attempts, the boot server declares it dead, and attempts to restart it. If that fails, it arranges to allocate a new pool processor on which a new copy is started. In this manner, critical services are rebooted automatically if they should ever fail. The boot server can itself be replicated, to guard against its own failure.

7.6.6. The TCP/IP Server

Although Amoeba uses the FLIP protocol internally to achieve high performance, sometimes it is necessary to speak TCP/IP, for example, to communicate with X terminals, to send and receive mail to non-Amoeba machines and to interact with other Amoeba systems via the Internet. To permit Amoeba to do these things, a TCP/IP server has been provided.

To establish a connection, an Amoeba process does an RPC with the TCP/IP server giving it a TCP/IP address. The caller is then blocked until the connection has been established or refused. In the reply, the TCP/IP server provides a capability for using the connection. Subsequent RPCs can send and receive packets from the remote machine without the Amoeba process having to know

that TCP/IP is being used. This mechanism is less efficient than FLIP, so it is used only when it is not possible to use FLIP.

7.6.7. Other Servers

Amoeba supports various other servers. These include a disk server (used by the directory server for storing its arrays of capability pairs), various other I/O servers, a time-of-day server, and a random number server (useful for generating ports, capabilities, and FLIP addresses). The so-called Swiss Army Knife server deals with many activities that have to be done later by starting up processes at a specified time in the future. Mail servers deal with incoming and outgoing electronic mail.

7.7. SUMMARY

Amoeba is a new operating system designed to make a collection of independent computers appear to its users as a single timesharing system. In general, the users are not aware of where their processes are running (or even on what type of CPU), and are not aware of where their files are stored or how many copies are being maintained for reasons of availability and performance. However, users who are explicitly interested in parallel programming can exploit the existence of multiple CPUs for splitting a single job over many machines.

Amoeba is based on a microkernel that handles low-level process and memory management, communication, and I/O. The file system and the rest of the operating system can run as user processes. This division of labor keeps the kernel small and simple.

Amoeba has a single mechanism for naming and protecting all objects—capabilities. Each capability contains rights telling which operations may be performed using it. Capabilities are protected cryptographically using one-way functions. Each one contains a checksum field that assures the security of the capability.

Three communication mechanisms are supported: RPC and raw FLIP for point-to-point communication, and reliable group communication for multiparty communication. The RPC guarantees at-most-once semantics. The group communication is based on reliable broadcasting as provided by the sequencer algorithm. Both mechanisms are supported on top of the FLIP protocol and are closely integrated. Raw FLIP is only used under special circumstances.

The Amoeba file system consists of three servers: the bullet server for file storage, the directory server for file naming, and the replication server for file replication. The bullet server maintains immutable files that are stored

contiguously on disk and in the cache. The directory server is a fault-tolerant server that maps ASCII strings to capabilities. The replication server handles lazy replication.

PROBLEMS

1. The Amoeba designers assumed that memory would soon be available in large amounts for low prices. What impact did this assumption have on the design?
2. State an advantage and a disadvantage of the processor pool model compared to the personal multiprocessor model.
3. List three functions of the Amoeba microkernel.
4. Some Amoeba servers can be run in the kernel as well as in user space. Their clients cannot tell the difference (except by timing them). What is it about Amoeba that makes it impossible for clients to tell the difference?
5. A malicious user is trying to guess the bullet server's get-port by picking a random 48-bit number, running it through the well-known one-way function, and seeing if the put-port comes out. It takes 1 msec per trial. How long will it take to guess the get-port, on the average?
6. How does a server tell that a capability is an owner capability, as opposed to a restricted capability? How are owner capabilities verified?
7. If a capability is not an owner capability, how do servers check it for validity?
8. Explain what a glocal variable is.
9. Why does the *trans* call have parameters for both sending and receiving? Would it not have been better and simpler to have two calls, *send_request* and *get_reply*, one for sending and one for receiving?
10. Amoeba claims to guarantee at-most-once semantics on RPCs. Suppose that three file servers offer the same service. A client does an RPC with one of them, which carries out the request and then crashes. Then the RPC is repeated with another server, resulting in the work being done twice. Is this possible? If so, what does the guarantee mean? If not, how is it prevented?
11. Why does the sequencer need a history buffer?

12. Two algorithms for broadcasting in Amoeba were presented in the text. In method 1, the sender sends a point-to-point message to the sequencer, which then broadcasts it. In method 2, the sender does the broadcast, with the sequencer then broadcasting a small acknowledgement packet. Consider a 10-Mbps network on which processing a packet-arrived interrupt takes 500 microsec, independent of the packet size. If all data packets are 1K bytes, and acknowledgement packets are 100 bytes, how much bandwidth and how much CPU time are consumed per 1000 broadcasts by the two methods?
13. What property of FLIP addressing makes it possible to handle process migration and automatic network reconfiguration in a straightforward way?
14. The bullet server supports immutable files for its users. Are the bullet server's own tables also immutable?
15. Why does the bullet server have uncommitted and committed files?
16. In Amoeba, links to a file can be created by putting capabilities with different rights in different directories. These give different users different permissions. This feature is not present in UNIX. Why?

8

Case Study 2: Mach

Our second example of a modern, microkernel-based operating system is Mach. We will start out by looking at its history and how it has evolved from earlier systems. Then we will examine in some detail the microkernel itself, focusing on processes and threads, memory management, and communication. Finally, we will discuss UNIX emulation. More information about Mach can be found in (Accetta et al., 1986; Baron et al., 1985; Black et al., 1992; Boykin et al., 1993; Draves et al., 1991; Rashid, 1986a; Rashid, 1986b; and Sansom et al., 1986).

3.1. INTRODUCTION TO MACH

In this section we will give a brief introduction to Mach. We will start with the history and goals. Then we will describe the main concepts of the Mach microkernel and the principal server that runs on the microkernel.

3.1.1. History of Mach

Mach's earliest roots go back to a system called **RIG (Rochester Intelligent Gateway)**, which began at the University of Rochester in 1975 (Ball et al., 1976). RIG was written for a 16-bit Data General minicomputer called the

Eclipse. Its main research goal was to demonstrate that operating systems could be structured in a modular way, as a collection of processes that communicated by message passing, including over a network. The system was designed and built, and indeed showed that such an operating system could be constructed.

When one of its designers, Richard Rashid, left the University of Rochester and moved to Carnegie-Mellon University in 1979, he wanted to continue developing message-passing operating systems but on more modern hardware. Various machines were considered. The machine selected was the PERQ, an early engineering workstation, with a bitmapped screen, mouse, and network connection. It was also microprogrammable. The new operating system for the PERQ was called **Accent**. It improved on RIG by adding protection, the ability to operate transparently over the network, 32-bit virtual memory, and other features. An initial version was up and running in 1981.

By 1984 Accent was being used on 150 PERQs but it was clearly losing out to UNIX. This observation led Rashid to begin a third-generation operating systems project called **Mach**. By making Mach compatible with UNIX, he hoped to be able to use the large volume of UNIX software becoming available. In addition, Mach had many other improvements over Accent, including threads, a better interprocess communication mechanism, multiprocessor support, and a highly imaginative virtual memory system.

Around this time, DARPA, the U.S. Department of Defense's Advanced Research Projects Agency, was hunting around for an operating system that supported multiprocessors as part of its Strategic Computing Initiative. CMU was selected, and with substantial DARPA funding, Mach was developed further. Initially, Mach consisted of a modified version of 4.1 BSD with additional features inserted for communication and memory management. As 4.2 BSD and 4.3 BSD became available, the Mach code was combined with them to give updated versions. Although this approach led to a large kernel, it did guarantee absolute compatibility with Berkeley UNIX, an important goal for DARPA.

The first version of Mach was released in 1986 for the VAX 11/784, a four-CPU multiprocessor. Shortly thereafter, ports to the IBM PC/RT and Sun 3 were done. By 1987, Mach was also running on the Encore and Sequent multiprocessors. Although Mach had networking facilities, at this time it was conceived of primarily as a single machine or multiprocessor system rather than as a transparent distributed operating system for a collection of machines on a LAN.

Shortly thereafter, the Open Software Foundation, a consortium of computer vendors led by IBM, DEC, and Hewlett Packard was formed in an attempt to wrest control of UNIX from its owner, AT&T, which was then working closely with Sun Microsystems to develop System V Release 4. The OSF members feared that this alliance would give Sun a competitive advantage over them. After some missteps, OSF chose Mach 2.5 as the basis for its first operating system, OSF/1. Although Mach 2.5 and OSF/1 contained large amounts of

Berkeley and AT&T code, the hope was that OSF would at least be able to control the direction in which UNIX was going.

As of 1988, the Mach 2.5 kernel was large and monolithic, due to the presence of a large amount of Berkeley UNIX code in the kernel. In 1988, CMU removed all the Berkeley code from the kernel and put it in user space. What remained was a microkernel consisting of pure Mach. In this chapter, we will focus on the Mach 3 microkernel and one user-level operating system emulator, for BSD UNIX. One difficulty, however, is that Mach is under development, so any description is at best a snapshot. Fortunately, most of the basic ideas discussed in this chapter are relatively stable, but some of the details may change in time.

8.1.2. Goals of Mach

Mach has evolved considerably since its first incarnation as RIG. The goals of the project have also changed as time has gone on. The current primary goals can be summarized as follows:

1. Providing a base for building other operating systems (e.g., UNIX).
2. Supporting large sparse address spaces.
3. Allowing transparent access to network resources.
4. Exploiting parallelism in both the system and the applications.
5. Making Mach portable to a larger collection of machines.

These goals encompass both research and development. The idea is to explore multiprocessor and distributed systems while being able to emulate existing systems, such as UNIX, MS-DOS, and the Macintosh operating system.

Much of the initial work on Mach concentrated on single- and multiprocessor systems. At the time Mach was designed, few systems had support for multiprocessors. Even now, few multiprocessor systems other than Mach are machine independent.

8.1.3. The Mach Microkernel

The Mach microkernel has been built as a base upon which UNIX and other operating systems can be emulated. This emulation is done by a software layer that runs outside the kernel, as shown in Fig. 8-1. Each emulator consists of a part that is present in its application programs' address space, as well as one or more servers that run independently from the application programs. It should be noted that multiple emulators can be running simultaneously, so it is possible to

run 4.3BSD, System V, and MS-DOS programs on the same machine at the same time.

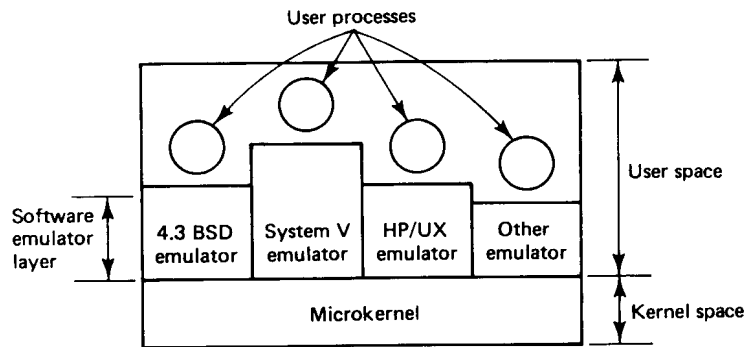


Fig. 8-1. The abstract model for UNIX emulation using Mach.

Like other microkernels, the Mach kernel, provides process management, memory management, communication, and I/O services. Files, directories, and other traditional operating system functions are handled in user space. The idea behind the Mach kernel is to provide the necessary mechanisms for making the system work, but leaving the policy to user-level processes.

The kernel manages five principal abstractions:

1. Processes.
2. Threads.
3. Memory objects.
4. Ports.
5. Messages.

In addition, the kernel manages several other abstractions either related to these or less central to the model.

A process is basically an environment in which execution can take place. It has an address space holding the program text and data, and usually one or more stacks. The process is the basic unit for resource allocation. For example, a communication channel is always “owned” by a single process.

As an aside, for the most part we will stick with the traditional nomenclature throughout this chapter, even though this means deviating from the terminology used in the Mach papers (e.g., Mach, of course, has processes, but they are called “tasks.”)

A thread in Mach is an executable entity. It has a program counter and a set

of registers associated with it. Each thread is part of exactly one process. A process with one thread is similar to a traditional (e.g., UNIX) process.

A concept that is unique to Mach is the **memory object**, a data structure that can be mapped into a process' address space. Memory objects occupy one or more pages and form the basis of the Mach virtual memory system. When a process attempts to reference a memory object that is not presently in physical main memory, it gets a page fault. As in all operating systems, the kernel catches the page fault. However, unlike other systems, the Mach kernel can send a message to a user-level server to fetch the missing page.

Interprocess communication in Mach is based on message passing. To receive messages, a user process asks the kernel to create a kind of protected mailbox, called a **port**, for it. The port is stored inside the kernel, and has the ability to queue an ordered list of messages. Queues are not fixed in size, but for flow control reasons, if more than n messages are queued on a port, a process attempting to send to it is suspended to give the port a chance to be emptied. The parameter n is settable per port.

A process can give the ability to send to (or receive from) one of its ports to another process. This permission takes the form of a **capability**, and includes not only a pointer to the port, but also a list of rights that the other process has with respect to the port (e.g., SEND right). Once this permission has been granted, the other process can send messages to the port, which the first process can then read. All communication in Mach uses this mechanism. Mach does not support a full capability mechanism; ports are the only objects for which capabilities exist.

8.1.4. The Mach BSD UNIX Server

As we described above, the Mach designers have modified Berkeley UNIX to run in user space, as an application program. This structure has a number of significant advantages over a monolithic kernel. First, by breaking the system up into a part that handles the resource management (the kernel) and a part that handles the system calls (the UNIX server), both pieces become simpler and easier to maintain. In a way, this split is somewhat reminiscent of the division of labor in IBM's mainframe operating system VM/370, in which the kernel simulates a collection of bare 370s, each of which runs a single-user operating system.

Second, by putting UNIX in user space, it can be made extremely machine independent, enhancing its portability to a wide variety of computers. All the machine dependencies can be removed from UNIX and hidden away inside the Mach kernel.

Third, as we mentioned earlier, multiple operating systems can run simultaneously. On a 386, for example, Mach can run a UNIX program and an MS-DOS

program at the same time. Similarly, it is possible to test a new experimental operating system and run a production operating system at the same time.

Fourth, real-time operation can be added to the system because all the traditional obstacles that UNIX presents to real-time work, such as disabling interrupts in order to update critical tables are either eliminated altogether or moved into user space. The kernel can be carefully structured not to have this type of hindrance to real-time applications.

Finally, this arrangement can be used to provide better security between processes, if need be. If each process has its own version of UNIX, it is very difficult for one process to snoop on the other one's files.

8.2. PROCESS MANAGEMENT IN MACH

Process management in Mach deals with processes, threads, and scheduling. In this section we will look at each of these in turn.

8.2.1. Processes

A process in Mach consists primarily of an address space and a collection of threads that execute in that address space. Processes are passive. Execution is associated with the threads. Processes are used for collecting all the resources related to a group of cooperating threads into convenient containers.

Figure 8-2 illustrates a Mach process. In addition to an address space and threads, it has some ports and other properties. The ports shown in the figure all have special functions. The **process port** is used to communicate with the kernel. Many of the kernel services that a process can request are done by sending a message to the process port, rather than making a system call. This mechanism is used throughout Mach to reduce the actual system calls to a bare minimum. A small number of them will be discussed in this chapter, to give an idea of what they are like.

In general, the programmer is not even aware of whether or not a service requires a system call. All services, including both those accessed by system calls and those accessed by message passing, have stub procedures in the library. It is these procedures that are described in the manuals and called by application programs. The procedures are generated from a service definition by the **MIG (Mach Interface Generator)** compiler.

The **bootstrap port** is used for initialization when a process starts up. The very first process reads the bootstrap port to learn the names of kernel ports that provide essential services. UNIX processes also use it to communicate with the UNIX emulator.

The **exception port** is used to report exceptions caused by the process.

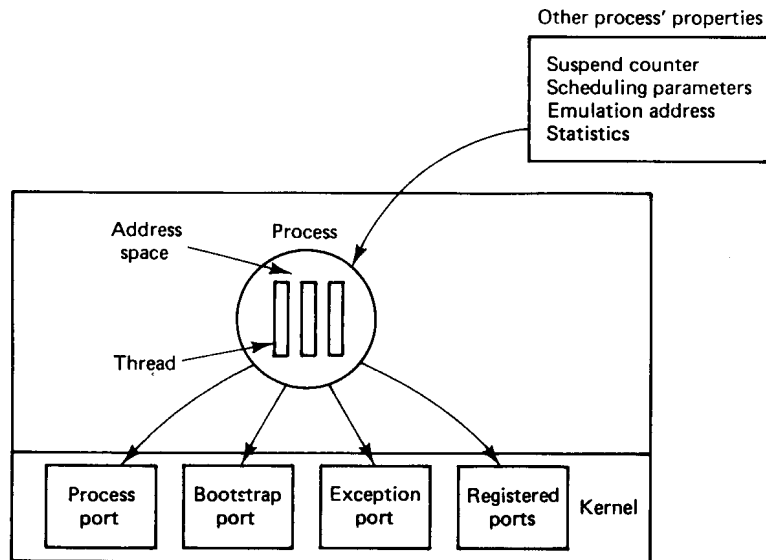


Fig. 8-2. A Mach process.

Typical exceptions are division by zero and illegal instruction executed. The port tells the system where the exception message should be sent. Debuggers also use the exception port.

The **registered ports** are normally used to provide a way for the process to communicate with standard system servers. For example, the name server makes it possible to present a string and get back the corresponding port for certain basic servers.

Processes also have other properties. A process can be runnable or blocked, independent of the state of its threads. If a process is runnable, those threads that are also runnable can be scheduled and run. If a process is blocked, its threads may not run, no matter what state they are in.

The per-process items also include scheduling parameters. These include the ability to specify which processors the process' threads can run on. This feature is most useful on a multiprocessor system. For example, the process can use this power to force each thread to run on a different processor, or to force them all to run on the same processor, or anything in between. In addition, each process has a default priority that is settable. When a thread is created, the new thread is given this priority. It is also possible to change the priority of all the existing threads.

Emulation addresses can be set to tell the kernel where in the process' address space system call handlers are located. The kernel needs to know these

addresses to handle UNIX system calls that need to be emulated. These are set once when the UNIX emulator is started up and are inherited by all of the emulator's children (i.e., all the UNIX processes).

Finally, every process has statistics associated with it, including the amount of memory consumed, the run times of the threads, and so on. A process that is interested in this information can acquire it by sending a message to the process port.

It is also worth mentioning what a Mach process does not have. A process does not have a uid, gid, signal mask, root directory, working directory, or file descriptor array, all of which UNIX processes do have. All of this information is managed by the emulation package, so the kernel knows nothing at all about it.

Process Management Primitives

Mach provides a small number of primitives for managing processes. Most of these are done by sending messages to the kernel via the process port, rather than actual system calls. The most important of these calls are shown in Fig. 8-3. These, like all calls in Mach, have prefixes indicating the group they belong to, but we have omitted these here (and in subsequent tables) for the sake of brevity.

Call	Description
Create	Create a new process, inheriting certain properties
Terminate	Kill a specified process
Suspend	Increment suspend counter
Resume	Decrement suspend counter. If it is 0, unblock the process
Priority	Set the priority for current or future threads
Assign	Tell which processor new threads should run on
Info	Return information about execution time, memory usage, etc.
Threads	Return a list of the process' threads

Fig. 8-3. Selected process management calls in Mach.

The first two calls in Fig. 8-3 are for creating and destroying processes, respectively. The process creation call specifies a prototype process, not necessarily the caller. The child is a copy of the prototype, except that the call has a parameter that tells whether or not the child is to inherit the parent's address

space. If it does not inherit the parent's address space, objects (e.g., text, initialized data, and a stack) can be mapped in later. Initially, the child has no threads. It does, however, automatically get a process port, a bootstrap port, and an exception port. Other ports are not inherited automatically since each port may have only one reader.

Processes can be suspended and resumed under program control. Each process has a counter, incremented by the *suspend* call and decremented by the *resume* call, that can block or unblock it. When the counter is 0, the process is able to run. When it is positive, it is suspended. Having a counter is more general than just having a bit, and helps avoid race conditions.

The *priority* and *assign* calls give the programmer control over how and where its threads run on multiprocessor systems. CPU scheduling is done using priorities, so the programmer has fine-grain control over which threads are most important and which are least important. The *assign* call makes it possible to control which thread runs on which CPU or group of CPUs.

The last two calls of Fig. 8-3 return information about the process. The former gives statistical information and the latter returns a list of all the threads.

8.2.2. Threads

The active entities in Mach are the threads. They execute instructions and manipulate their registers and address spaces. Each thread belongs to exactly one process. A process cannot do anything unless it has one or more threads.

All the threads in a process share the address space and all the process-wide resources shown in Fig. 8-2. Nevertheless, threads also have private per-thread resources. One of these is the **thread port**, which is analogous to the process port. Each thread has its own thread port, which it uses to invoke thread-specific kernel services, such as exiting when the thread is finished. Since ports are process-wide resources, each thread has access to its siblings' ports, so each thread can control the others if need be.

Mach threads are managed by the kernel, that is, they are what are sometimes called heavyweight threads rather than lightweight threads (pure user space threads). Thread creation and destruction are done by the kernel and involve updating kernel data structures. They provide the basic mechanisms for handling multiple activities within a single address space. What the user does with these mechanisms is up to the user.

On a single CPU system, threads are timeshared, first one running, then another. On a multiprocessor, several threads can be active at the same time. This parallelism makes mutual exclusion, synchronization, and scheduling more important than they normally are, because performance now becomes a major issue, along with correctness. Since Mach is intended to run on multiprocessors, these issues have received special attention.

Like a process, a thread can be runnable or blocked. The mechanism is similar, too: a counter per thread that can be incremented and decremented. When it is zero, the thread is runnable. When it is positive, the thread must wait until another thread lowers it to zero. This mechanism allows threads to control each other's behavior.

A variety of primitives is provided. The basic kernel interface provides about two dozen thread primitives, many of them concerned with controlling scheduling in detail. On top of these primitives one can build various thread packages.

Mach's approach is the **C threads** package. This package is intended to make the kernel thread primitives available to users in a simple and convenient form. It does not have the full power that the kernel interface offers, but it is enough for the average garden-variety programmer. It has also been designed to be portable to a wide variety of operating systems and architectures.

The C threads package provides sixteen calls for direct thread manipulation. The most important ones are listed in Fig. 8-4. The first one, *fork*, creates a new thread in the same address space as the calling thread. It runs the procedure specified by a parameter rather than the parent's code. After the call, the parent thread continues to run in parallel with the child. The thread is started with a priority and on a processor determined by the process' scheduling parameters, as discussed above.

Call	Description
Fork	Create a new thread running the same code as the parent thread
Exit	Terminate the calling thread
Join	Suspend the caller until a specified thread exits
Detach	Announce that the thread will never be jointed (waited for)
Yield	Give up the CPU voluntarily
Self	Return the calling thread's identity to it

Fig. 8-4. The principal C threads calls for direct thread management.

When a thread has done its work, it calls *exit*. If the parent is interested in waiting for the thread to finish, it can call *join* to block itself until a specific child thread terminates. If the thread has already terminated, the parent continues immediately. These three calls are roughly analogous to the FORK, EXIT, and WAITPID system calls in UNIX.

The fourth call, *detach*, does not exist in UNIX. It provides a way to announce that a particular thread will never be waited for. If that thread ever

exits, its stack and other state information will be deleted immediately. Normally, this cleanup happens only after the parent has done a successful *join*. In a server, it might be desirable to start up a new thread to service each incoming request. When it has finished, the thread exits. Since there is no need for the initial thread to wait for it, the server thread should be detached.

The *yield* call is a hint to the scheduler that the thread has nothing useful to do at the moment, and is waiting for some event to happen before it can continue. An intelligent scheduler will take the hint and run another thread. In Mach, which normally schedules its threads preemptively, *yield* is only optimization. In systems that have nonpreemptive scheduling, it is essential that a thread that has no work to do release the CPU, to give other threads a chance to run.

Finally, *self* returns the caller's identity, analogous to *GETPID* in UNIX.

The remaining calls (not shown in the figure), allow threads to be named, allow the program to control the number of threads and the sizes of their stacks, and provide interfaces to the kernel threads and message-passing mechanism.

Synchronization is done using mutexes and condition variables. The mutex primitives are *lock*, *trylock*, and *unlock*. Primitives are also provided to allocate and free mutexes. Mutexes work like binary semaphores, providing mutual exclusion, but not conveying information.

The operations on condition variables are *signal*, *wait*, and *broadcast*, which are used to allow threads to block on a condition and later be awakened when another thread has caused that condition to occur.

Implementation of C Threads in Mach

Various implementations of C threads are available on Mach. The original one did everything in user space inside a single process. This approach timeshared all the C threads over one kernel thread, as shown in Fig. 8-5(a). This approach can also be used on UNIX or any other system that provides no kernel support. The threads were run as coroutines, which means that they were scheduled nonpreemptively. A thread could keep the CPU as long as it wanted or was able to. For the producer-consumer problem, the producer would eventually fill the buffer and then block, giving the consumer a chance to run. For other applications, however, threads had to call *yield* from time to time to give other threads a chance.

The original implementation package suffers from a problem inherent to most user-space threads packages that have no kernel support. If one thread makes a blocking system call, such as reading from the terminal, the whole process is blocked. To avoid this situation, the programmer must avoid blocking system calls. In Berkeley UNIX, there is a call *SELECT* that can be used to tell whether any characters are pending, but the whole situation is quite messy.

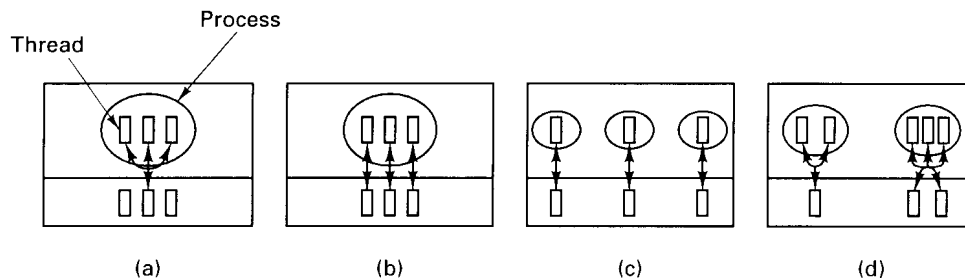


Fig. 8-5. (a) All C threads use one kernel thread. (b) Each C thread has its own kernel thread. (c) Each C thread has its own single-threaded process. (d) Arbitrary mapping of user threads to kernel threads.

A second implementation is to use one Mach thread per C thread, as shown in Fig. 8-5(b). These threads are scheduled preemptively. Furthermore, on a multiprocessor, they may actually run in parallel, on different CPUs. In fact, it is also possible to multiplex m user threads on n kernel threads, although the most common case is $m = n$.

A third implementation package has one thread per process, as shown in Fig. 8-5(c). The processes are set up so that their address spaces all map onto the same physical memory, allowing sharing in the same way as in the previous implementations. This implementation is only used when specialized virtual memory usage is required. The method has the drawback that ports, UNIX files, and other per-process resources cannot be shared, limiting its value appreciably.

The fourth package allows an arbitrary number of user threads to be mapped onto an arbitrary number of kernel threads, as shown in Fig. 8-5(d).

The main practical value of the first approach is that because there is no true parallelism, successive runs give reproducible results, allowing easier debugging. The second approach has the advantage of simplicity and was used for a long time. The third one is not normally used. The fourth one, although the most complicated, gives the greatest flexibility and is the one normally used at present.

8.2.3. Scheduling

Mach scheduling has been heavily influenced by its goal of running on multiprocessors. Since a single-processor system is effectively a special case of a multiprocessor (with only one CPU), our discussion will focus on scheduling in multiprocessor systems. For more information, see (Black, 1990).

The CPUs in a multiprocessor can be assigned to **processor sets** by software. Each CPU belongs to exactly one processor set. Threads can also be

assigned to processor sets by software. Thus each processor set has a collection of CPUs at its disposal and a collection of threads that need computing power. The job of the scheduling algorithm is to assign threads to CPUs in a fair and efficient way. For purposes of scheduling, each processor set is a closed world, with its own resources and its own customers, independent of all the other processor sets.

This mechanism gives processes a large amount of control over their threads. A process can assign an important thread to a processor set with one CPU and no other threads, thus ensuring that the thread runs all the time. It can also dynamically reassign threads to processor sets as the work proceeds, keeping the load balanced. While the average compiler is not likely to use this facility, a data base management system or a real-time system might well use it.

Thread scheduling in Mach is based on priorities. Priorities are integers from 0 to some maximum (usually 31 or 127), with 0 being the highest priority and 31 or 127 being the lowest priority. This priority reversal comes from UNIX. Each thread has three priorities assigned to it. The first priority is a base priority, which the thread can set itself, within certain limits. The second priority is the lowest numerical value that the thread may set its base priority to. Since using a higher value gives worse service, a thread will normally set its value to the lowest value it is permitted, unless it is trying intentionally to defer to other threads. The third priority is the current priority, used for scheduling purposes. It is computed by the kernel by adding to the base priority a function based on the thread's recent CPU usage.

Mach threads are visible to the kernel, at least when the model of Fig. 8-5(b) is used. Each thread competes for CPU cycles with all other threads, without regard to which thread is in which process. When making scheduling decisions, the kernel does not take into account which thread belongs to which process.

Associated with each processor set is an array of run queues, as shown in Fig. 8-6. The array has 32 queues, corresponding to threads currently at priorities 0 through 31. When a thread at priority n becomes runnable, it is put at the end of queue n . A thread that is not runnable is not present on any run queue.

Each run queue has three variables attached to it. The first is a mutex that is used to lock the data structure. It is used to make sure that only one CPU at a time is manipulating the queues. The second variable is the count of the number of threads on all the queues combined. If this count becomes 0, there is no work to do. The third variable is a hint as to where to find the highest-priority thread. It is guaranteed that no thread is at a higher priority, but the highest may be at a lower priority. This hint allows the search for the highest-priority thread to avoid the empty queues at the top.

In addition to the global run queues shown in Fig. 8-6, each CPU has its own local run queue. Each local run queue holds those threads that are permanently bound to that CPU, for example, because they are device drivers for I/O devices

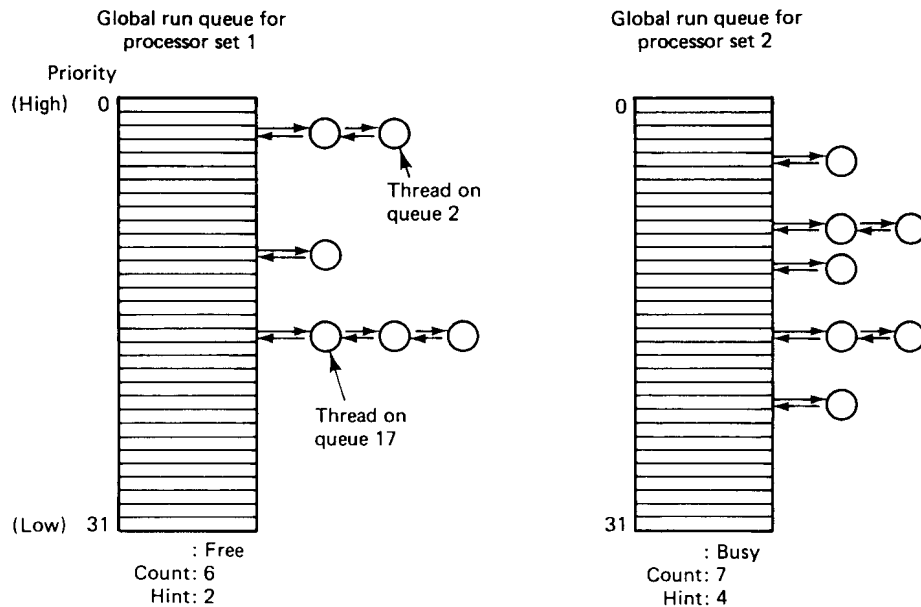


Fig. 8-6. The global run queues for a system with two processor sets.

attached to that CPU. These threads can run on only one CPU, so putting them on the global run queue is incorrect (because the “wrong” CPU might choose them).

We can now describe the basic scheduling algorithm. When a thread blocks, exits, or uses up its quantum, the CPU it is running on first looks on its local run queue to see if there are any active threads. This check merely requires inspecting the count variable associated with the local run queue. If it is nonzero, the CPU begins searching the queue for the highest-priority thread, starting at the queue specified by the hint. If the local run queue is empty, the same algorithm is applied to the global run queue, the only difference being that the global run queue must be locked before it can be searched. If there are no threads to run on either queue, a special idle thread is run until some thread becomes ready.

If a runnable thread is found, it is scheduled and run for one quantum. At the end of the quantum, both the local and global run queues are checked to see if any other threads at its priority or higher are runnable, with the understanding that all threads on the local run queue have higher priority than all threads on the global run queue. If a suitable candidate is found, a thread switch occurs. If not, the thread is run for another quantum. Threads may also be preempted. On multiprocessors, the length of the quantum is variable, depending on the number

of threads that are runnable. The more runnable threads and the fewer CPUs there are, the shorter the quantum. This algorithm gives good response time to short requests, even on heavily loaded systems, but provides high efficiency (i.e., long quanta) on lightly loaded systems.

On every clock tick, the CPU increments the priority counter of the currently running thread by a small amount. As the value goes up, the priority goes down and the thread will eventually move to a higher-numbered (i.e., lower-priority) queue. The priority counters are lowered by the passage of time.

For some applications, a large number of threads may be working together to solve a single problem, and it may be important to control the scheduling in detail. Mach provides a hook to give threads some additional control over their scheduling (in addition to the processor sets and priorities). The hook is a system call that allows a thread to lower its priority to the absolute minimum for a specified number of seconds. Doing so gives other threads a chance to run. When the time interval is over, the priority is restored to its previous value.

This system call has another interesting property: it can name its successor if it wants to. For example, after sending a message to another thread, the sending thread can give up the CPU and request that the receiving thread be allowed to run next. This mechanism, called **handoff scheduling**, bypasses the run queues entirely. Used wisely, it can enhance performance. The kernel also uses it in some circumstances, as an optimization.

Mach can be configured to do affinity scheduling, but generally this option is off. When it is on, the kernel schedules a thread on the CPU it last ran on, in hopes that part of its address space is still in that CPU's cache. Affinity scheduling is only applicable to multiprocessors.

Finally, several other scheduling algorithms are supported in some versions, including algorithms useful for real-time applications.

8.3. MEMORY MANAGEMENT IN MACH

Mach has a powerful, elaborate, and highly flexible memory management system based on paging, including features found in few other operating systems. In particular, it separates the machine-independent parts of the memory management system from the machine-dependent parts in an extremely clear and unusual way. This separation makes the memory management far more portable than in other systems. In addition, the memory management system interacts closely with the communication system, which we will discuss in the following section.

The aspect of Mach's memory management that sets it apart from all others is that the code is split into three parts. The first part is the *pmap* module, which runs in the kernel and is concerned with managing the MMU. It sets up the

MMU registers and hardware page tables, and catches all page faults. This code depends on the MMU architecture and must be rewritten for each new MMU Mach has to support. The second part, the machine-independent kernel code, is concerned with processing page faults, managing address maps, and replacing pages.

The third part of the memory management code runs as a user process called a **memory manager** or sometimes an **external pager**. It handles the logical (as opposed to physical) part of the memory management system, primarily management of the backing store (disk). For example, keeping track of which virtual pages are in use, which are in main memory, and where pages are kept on disk when they are not in main memory are all done by the memory manager.

The kernel and the memory manager communicate through a well-defined protocol, making it possible for users to write their own memory managers. This division of labor gives users the ability to implement special-purpose paging systems in order to write systems with special requirements. It also has the potential for making the kernel smaller and simpler by moving a large section of the code out into user space. On the other hand, it has the potential for making it more complicated, since the kernel must protect itself from buggy or malicious memory managers, and with two active entities involved in handling memory, there is now the danger of race conditions.

8.3.1. Virtual Memory

The conceptual model of memory that Mach user processes see is a large, linear virtual address space. For most 32-bit CPU chips, the user address space runs from address 0 to address $2^{31} - 1$ because the kernel uses the top half for its own purposes. The address space is supported by paging. Since paging was designed to give the illusion of ordinary memory, but more of it than there really is, in principle there should be nothing else to say about how Mach manages virtual address space.

In reality, there is a great deal more to say. Mach provides a great deal of fine-grained control over how the virtual pages are used (for processes that are interested in that). To start with, the address space can be used in a sparse way. For example, a process might have dozens of sections of the virtual address space in use, each many megabytes from its nearest neighbor, with large holes of unused addresses between the sections.

Theoretically, any virtual address space can be used this way, so the ability to use a number of widely scattered sections is not really a property of the virtual address space architecture. In other words, any 32-bit machine should allow a process to have a 50K section of data spaced every 100 megabytes, from 0 to the 4-gigabyte limit. However, in many implementations, a linear page table from 0 to the highest used page is kept in kernel memory. On a machine with a

1K page size, this configuration requires 4 million page table entries, making it expensive, if not impossible. Even with a multilevel page table, such sparse usage is inconvenient at best. With Mach, the intention is to fully support sparse address spaces.

To determine which virtual addresses are in use and which are not, Mach provides a way to allocate and deallocate sections of virtual address space, called **regions**. The allocation call can specify a base address and a size, in which case the indicated region is allocated, or it can just specify a size, in which case the system finds a suitable address range and returns its base address. A virtual address is valid only if it falls in an allocated region. An attempt to use an address between allocated regions results in a trap, which, however, can be caught by the process if it so desires.

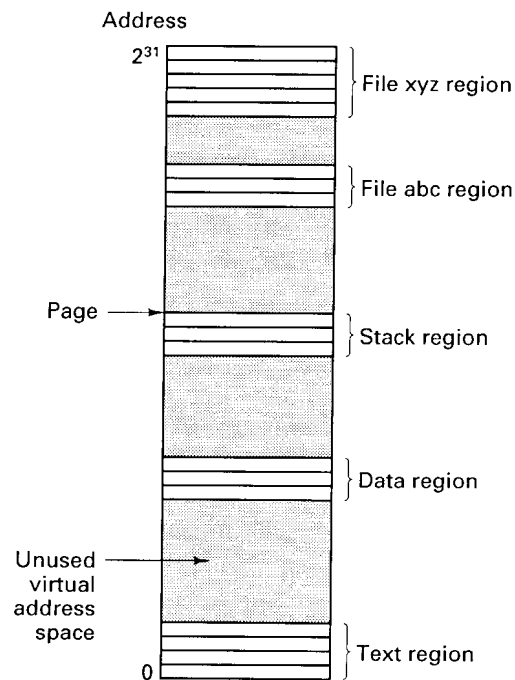


Fig. 8-7. An address space with allocated regions, mapped objects, and unused addresses.

A key concept relating to the use of virtual address space is the **memory object**. A memory object can be a page or a set of pages, but it can also be a file or other, more specialized data structure. A memory object can be mapped into an unused portion of the virtual address space, forming a new region, as shown in Fig. 8-7. When a file is mapped into the virtual address space, it can be read and written by normal machine instructions. Mapped files are paged in the usual

way. When a process terminates, its mapped files automatically appear back in the file system, complete with all the changes that were made to them when they were mapped in. It is also possible to unmap files or other memory objects explicitly, freeing their virtual addresses and making them available for subsequent allocation or mapping.

As an aside, file mapping is not the only way to access files. They can also be read the conventional way. However, even then, the library may map the files behind the user's back rather than reading them using the I/O system. Doing so allows the file pages to use the virtual memory system, rather than using dedicated buffers elsewhere in the system.

Mach supports a number of calls for manipulating virtual address spaces. The main ones are listed in Fig. 8-8. None are true system calls. Instead, they all write messages to the caller's process port.

Call	Description
Allocate	Make a region of virtual address space usable
Deallocate	Invalidate a region of virtual address space
Map	Map a memory object into the virtual address space
Copy	Make a copy of a region at another virtual address
Inherit	Set the inheritance attribute for a region
Read	Read data from another process' virtual address space
Write	Write data to another process' virtual address space

Fig. 8-8. Selected Mach calls for managing virtual memory.

The first call, *allocate*, makes a region of virtual address space usable. A process may inherit allocated virtual address space and it may allocate more, but any attempt to reference an unallocated address will fail. The second call, *deallocate*, invalidates a region (i.e., removes it from the memory map), thus making it possible to allocate it again or map something into it, using the *map* call.

The *copy* call copies a memory object onto a new region. The original remains unchanged. In this way, a single memory object can appear multiple times in the address space. Conceptually, calling *copy* is no different than having the object copied by a programmed loop. However *copy* is implemented in an optimized way, using shared pages, to avoid physical copying.

The *inherit* call affects the way that regions are inherited when new processes are created. The address space can be set up so that some regions are inherited and others are not. It will be discussed in the next section.

The *read* and *write* calls allow a thread to access virtual memory belonging to another process. These calls require the caller to have possession of the process port belonging to the remote process, something that process can pass to its friends if it wants to.

In addition to the calls listed in Fig. 8-8, a few other calls also exist. These calls are concerned primarily with getting and setting attributes, protection modes, and various kinds of statistical information.

8.3.2. Memory Sharing

Sharing plays an important role in Mach. No special mechanism is needed for the threads in a process to share objects: they all see the same address space automatically. If one of them has access to a piece of data, they all do. More interesting is the possibility of two or more processes sharing the same memory objects, or just sharing data pages, for that matter. Sometimes sharing is important on single CPU systems. For example, in the classical producer-consumer problem, it may be desirable to have the producer and consumer be different processes, yet share a common buffer so that the producer can put data into the buffer and the consumer can take data out of it.

On multiprocessor systems, sharing of objects between two or more processes is frequently even more important. In many cases, a single problem is being solved by a collection of cooperating processes running in parallel on different CPUs (as opposed to being timeshared on a single CPU). These processes may need access to buffers, tables, or other data structures continuously, in order to do their work. It is essential that the operating system allow this sharing to take place. Early versions of UNIX did not have this ability, for example, although it was added later.

Consider, for example, a system that analyzes digitized satellite images of the earth in real time, as they are transmitted to the ground. Such analysis is time consuming, and the same picture has to be examined for use in weather forecasting, predicting crop harvests, and tracking pollution. As each picture is received, it is stored as a file.

A multiprocessor is available to do the analysis. Since the meteorological, agricultural, and environmental programs are all quite different, and were written by different people, it is not reasonable to make them threads of the same process. Instead, each is a separate process, and each maps the current photograph into its address space, as shown in Fig. 8-9. Note that the file containing the photograph may be mapped in at a different virtual address in each process. Although each page is present in memory only once, it may appear in each process' page map at a different place. In this manner, all three processes can work on the same file at the same time in a convenient way.

Another important use of sharing is process creation. As in UNIX, in Mach

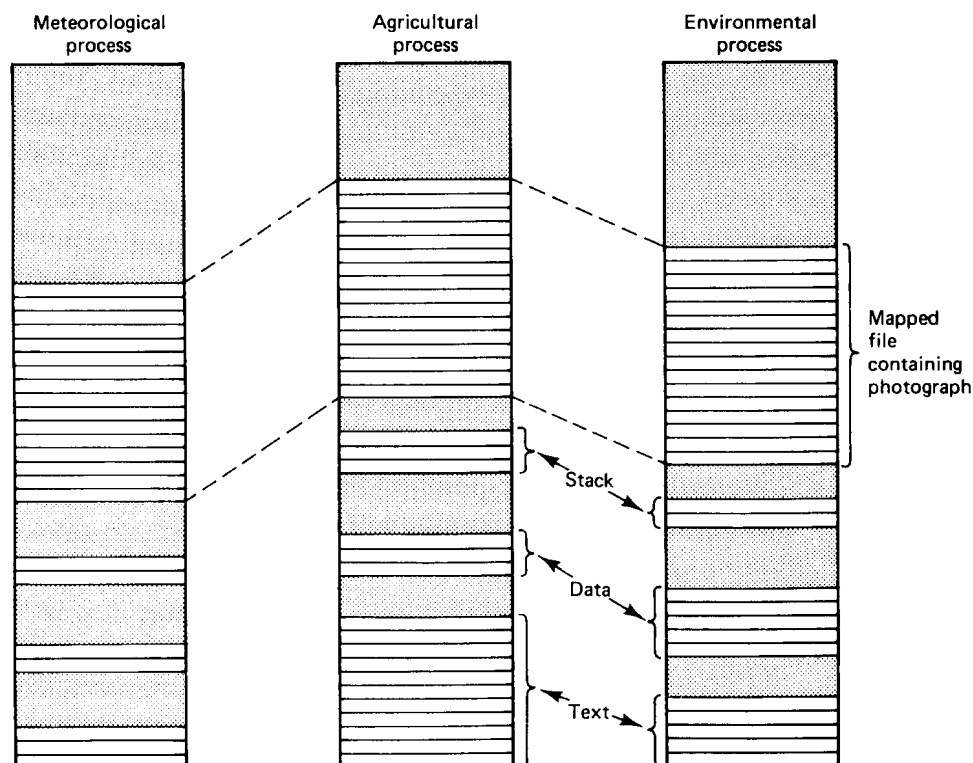


Fig. 8-9. Three processes sharing a mapped file.

the basic way for a new process to be created is as a copy of an existing process. In UNIX, a copy is always a clone of the process executing the FORK system call, whereas in Mach the child can be a clone of a different process (the prototype). Either way, the child is a copy of some other process.

One way to create the child is to copy all the pages needed and map the copies into the child's address space. Although this method is valid, it is unnecessarily expensive. The program text is normally read-only, so it cannot change, and parts of the data may also be read-only. There is no reason to copy read-only pages, since mapping them into both processes will do the job. Writable pages cannot always be shared because the semantics of process creation (at least in UNIX) say that although at the moment of creation the parent and child are identical, subsequent changes to either one are not visible in the other's address space.

In addition, some regions (e.g., certain mapped files) may not be needed in the child. Why go to a lot of trouble to arrange for them to be present in the child if they are not needed there?

To achieve these various goals, Mach allows processes to assign an **inheritance attribute** to each region in its address space. Different regions may have different attributes. Three values are provided:

1. The region is unused in the child process.
2. The region is shared between the prototype process and the child.
3. The region in the child process is a copy of the prototype.

If a region has the first value, the corresponding region in the child is unallocated. References to it are treated as references to any other unallocated memory—they generate traps. The child is free to allocate the region for its own purposes or to map a memory object there.

The second option is true sharing. The pages of the region are present in both the prototype's address space and the child's. Changes made by either one are visible to the other. This choice is not used for implementing the FORK system call in UNIX, but is frequently useful for other purposes.

The third possibility is to copy all the pages in the region and map the copies into the child's address space. FORK uses this option. Actually, Mach does not really copy the pages but uses a clever trick called **copy-on-write** instead. It places all the necessary pages in the child's virtual memory map, but marks them all read-only, as illustrated in Fig. 8-10. As long as the child makes only read references to these pages, everything works fine.

However, if the child attempts to write on any page, a protection fault occurs. The operating system then makes a copy of the page and maps the copy into the child's address space, replacing the read-only page that was there. The new page is marked read-write. In Fig. 8-10(b), the child has attempted to write to page 7. This action has resulted in page 7 being copied to page 8, and page 8 being mapped into the address space in place of page 7. Page 8 is marked read-write, so subsequent writes do not trap.

Copy-on-write has several advantages over doing all the copying at the time the new process is created. First, some pages are read-only, so there is no need to copy them. Second, other pages may never be referenced, so even if they are potentially writable, they do not have to be copied. Third, still other pages may be writable, but the child may deallocate them rather than using them. Here too, avoiding a copy is worthwhile. In this manner, only those pages that the child actually writes on have to be copied.

Copy-on-write also has some disadvantages. For one thing, the administration is more complicated, since the system must keep track of the fact that some pages are genuinely read-only, with a write being a programming error, whereas other pages are to be copied if written. For another, copy-on-write requires multiple kernel traps, one for each page that is ultimately written. Depending on the

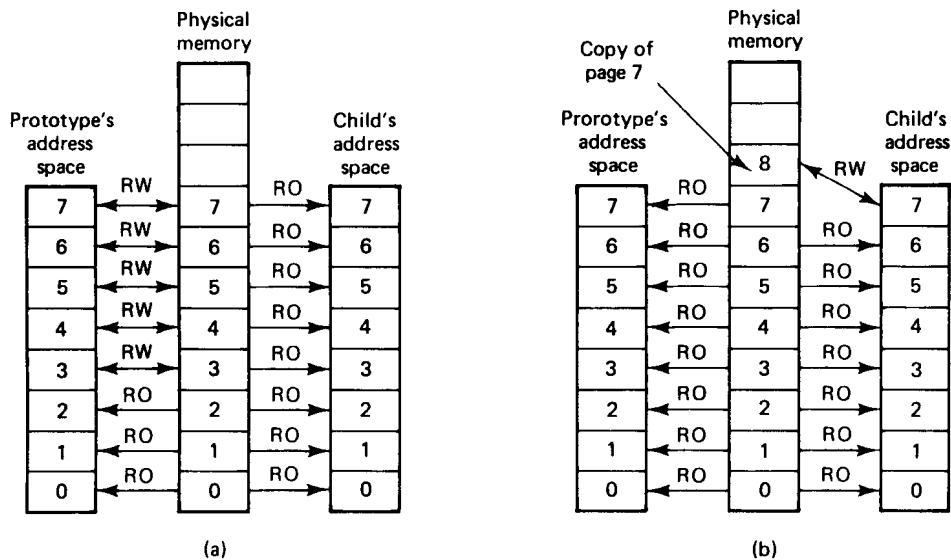


Fig. 8-10. Operation of copy-on-write. (a) After the FORK, all the child's pages are marked read-only. (b) When the child writes page 7, a copy is made.

hardware, one kernel trap followed by a multipage copy may not be that much more expensive than multiple kernel traps, each followed by a one-page copy. Finally, copy-on-write does not work over a network. Physical transport is always needed.

8.3.3. External Memory Managers

At the start of our discussion on memory management in Mach we briefly mentioned the existence of user-level memory managers. Let us now take a deeper look at them. Each memory object that is mapped in a process' address space must have an external memory manager that controls it. Different classes of memory objects are handled by different memory managers. Each of these can implement its own semantics, can determine where to store pages that are not in memory, and can provide its own rules about what becomes of objects after they are mapped out.

To map an object into a process' address space, the process sends a message to a memory manager asking it to do the mapping. Three ports are needed to do the job. The first, the **object port**, is created by the memory manager and will later be used by the kernel to inform the memory manager about page faults and other events relating to the object. The second, the **control port**, is created by

the kernel itself so that the memory manager can respond to these events (many require some action on the memory manager's part). The use of distinct ports is due to the fact that ports are unidirectional. The object port is written by the kernel and read by the memory manager; the control port works the other way around. The third port, the **name port**, is used as a kind of name to identify the object. For example, a thread can give the kernel a virtual address and ask which region it belongs to. The answer is a pointer to the name port. If addresses belong to the same region, they will be identified by the same name port.

When the memory manager maps in an object, it provides the capability for the object port as one of the parameters. The kernel then creates the other two ports and sends an initial message to the object port telling it about the control and name ports. The memory manager then sends back a reply telling the kernel what the object's attributes are, and informing it whether or not to keep the object in its cache after it is unmapped. Initially, all the object's pages are marked as unreadable/unwritable, to force a trap on the first use.

At this point the memory manager does a read on the object port and blocks. The memory manager remains idle until the kernel requests it to do something by writing a message to the object port. The thread that mapped the object in is now unblocked and allowed to execute.

Sooner or later, the thread will undoubtedly attempt to read or write a page belonging to the memory object. This operation will cause a page fault and a trap to the kernel. The kernel will then send a message to the memory manager via the object port, telling it which page has been referenced and asking it to please provide the page. This message is asynchronous because the kernel does not dare to block any of its threads waiting for a user process that may not reply. While waiting for a reply, the kernel suspends the faulting thread and looks for another thread to run.

When the memory manager hears about the page fault, it checks to see if the reference is legal. If not, it sends the kernel an error message. If it is legal, the memory manager gets the page by whatever method is appropriate for the object in question. If the object is a file, the memory manager seeks to the correct address and reads the page into its own address space. It then sends a reply back to the kernel providing a pointer to the page. The kernel maps the page into the faulting thread's address space. The thread can now be unblocked. This process is repeated as often as necessary to load all the pages needed.

To make sure that there is a steady supply of free page frames, a paging daemon thread in the kernel wakes up from time to time and checks the state of memory. If there are not enough free page frames, it picks a page to free using the second-chance algorithm. If the page is clean, it is normally just discarded. If the page is dirty, the daemon sends it to the memory manager in charge of the page's object. The memory manager is expected to write the page to disk and

tell when it is done. If the page belongs to a file, the memory manager will first seek to the page's offset in the file, then write it there.

Pages can be marked as **precious**, in which cases they will never be just discarded, even if they are clean. They will always be returned to their memory manager. Precious pages can be used, for example, for pages shared over the network for which there is only one copy which must never be discarded. Communication can also be initiated by the memory manager, for example, when a SYNC system call is done to flush the cache back to disk.

It is worth noting that the paging daemon is part of the kernel. Although the page replacement algorithm is completely machine independent, with a memory full of pages owned by different memory managers, there is no suitable way to let one of them decide which page to evict. The only method that might be possible is to statically partition the page frames among the various managers and let each one do page replacement on its set. However, since global algorithms are generally more efficient than local ones, this approach was not taken. Subsequent work has investigated this subject (Harty and Cheriton, 1992; and Subramian, 1991).

In addition to the memory managers for mapped files and other specialized objects, there is also a default memory manager for "ordinary" paged memory. When a process allocates a region of virtual address space using the *allocate* call, it is in fact mapping an object managed by the default manager. This manager provides zero-filled pages as needed. It uses a temporary file for swap space, rather than a separate swap area as UNIX does.

To make the idea of an external memory manager work, a strict protocol must be used for communication between the kernel and the memory managers. This protocol consists of a small number of messages that the kernel can send to a memory manager, and a small number of replies the memory manager can send back to the kernel. All communication is initiated by the kernel in the form of an asynchronous message on an object port for some memory object. Later, the memory manager sends an asynchronous reply on the control port.

Figure 8-11 lists the principal message types that the kernel sends to memory managers. When an object is mapped in using the *map* call of Fig. 8-8, the kernel sends an *init* message to the appropriate memory manager to let it initialize itself. The message specifies the ports to be used for discussing the object later. Requests from the kernel to ask for a page and deliver a page use *data_request* and *data_write* respectively. These handle the page traffic in both directions, and as such are the most important calls.

Data_unlock is a request from the kernel for the memory manager to unlock a locked page so that the kernel can use it for another process. *Lock_completed* signals the end of a *lock_request* sequence, and will be described below. Finally, *terminate* tells the memory manager that the object named in the message is no longer in use and can be removed from memory. Some calls that are

Call	Description
Init	Initialize a newly mapped-in memory object
Data_request	Give kernel a specific page to handle a page fault
Data_write	Take a page from memory and write it out
Data_unlock	Unlock a page so kernel can use it
Lock_completed	Previous Lock_request has been completed
Terminate	Be informed that this object is no longer in use

Fig. 8-11. Selected message types from the kernel to the external memory managers.

specific to the default memory manager also exist, as well as a few managing attributes and error handling.

The messages in Fig. 8-11 go from the kernel to the memory manager. The replies listed in Fig. 8-12 go the other way, from the memory manager back to the kernel. They are replies that the memory manager can use to respond to the above requests.

Call	Description
Set_attributes	Reply to Init
Data_provided	Here is the requested page (Reply to Data_request)
Data_unavailable	No page is available (Reply to Data_request)
Lock_request	Ask kernel to clean, flush, or lock pages
Destroy	Destroy an object that is no longer needed

Fig. 8-12. Selected message types from the external memory managers to the kernel.

The first one, *set_attributes*, is a reply to *init*. It tells the kernel that it is ready to handle a newly mapped-in object. The reply also provides mode bits for the object and tells the kernel whether or not to cache the object, even if no process currently has it mapped in. The next two are replies to *data_request*. That call asks the memory manager to provide a page. Which reply it gives depends on whether or not it can provide the page. The former supplies the page; the latter does not.

Lock_request allows the memory manager to ask the kernel to make certain pages clean, that is, send it the pages so that they can be written to disk. This

call can also be used to change the protection mode on pages (read, write, execute). Finally, *destroy* is used to tell the kernel that a certain object is no longer needed.

It is worth noting that when the kernel sends a message to a memory manager, it is effectively making an upcall. Although flexibility is gained this way, some system designers consider it inelegant for the kernel to call user programs to perform services for it. These people usually believe in hierarchical systems, with the lower layers providing services to the upper layers, not vice versa.

8.3.4. Distributed Shared Memory in Mach

The Mach external memory manager concept lends itself well to implementing a page-based distributed shared memory. In this section we will briefly describe some of the work done in this area. For more details, see (Forin et al., 1989). To review the basic concept, the idea is to have a single, linear, virtual address space that is shared among processes running on computers that do not have any physical shared memory. When a thread references a page that it does not have, it causes a page fault. Eventually, the page is located and shipped to the faulting machine, where it is installed so that the thread can continue executing.

Since Mach already has memory managers for different classes of objects, it is natural to introduce a new memory object, the shared page. Shared pages are managed by one or more special memory managers. One possibility is to have a single memory manager that handles all shared pages. Another is to have a different one for each shared page or collection of shared pages, to spread the load around.

Still another possibility is to have different memory managers for pages with different semantics. For example, one memory manager could guarantee complete memory coherence, meaning that any read following a write always sees the most recent data. Another memory manager could offer weaker semantics, for example, that a read never returns data that are more than 30 seconds out of date.

Let us consider the most basic case: one shared page, centralized control, and complete memory coherence. All other pages are local to a single machine. To implement this model, we need one memory manager that serves all the machines in the system. Let us call it the DSM (Distributed Shared Memory) server. The DSM server handles references to the shared page. Conventional memory managers handle the other pages. Up until now we have tacitly assumed that the memory manager or managers that service a machine must be local to that machine. In fact, because communication is transparent in Mach, a memory manager need not reside on the machine whose memory it is managing.

The shared page is always either readable or writable. If it is readable, it may be replicated on multiple machines. If it is writable, there is only one copy. The DSM server always knows the state of the shared page as well as which machine or machines it is currently on. If the page is readable, DSM has a valid copy itself.

Suppose that the page is readable and a thread somewhere tries to read it. The DSM server just sends that machine a copy, updates its tables to indicate one more reader, and is finished. The page will be mapped in on the new machine for reading.

Now suppose that one of the readers tries to write the page. The DSM server sends a message to the kernel or kernels that have the page asking for it back. The page itself need not be transferred, because the DSM server has a valid copy itself. All that is needed is an acknowledgement that the page is no longer in use. When all the kernels have released the page, the writer is given a copy along with exclusive permission to use it (for writing).

If somebody else now wants the page (when it is writable), the DSM server tells the current owner to stop using it and to send it back. When the page arrives, it can be given to one or more readers or one writer. Many variations on this centralized algorithm are possible, such as not asking for a page back until the machine currently using it has had it for some minimum time. A distributed solution is also possible.

8.4. COMMUNICATION IN MACH

The goal of communication in Mach is to support a variety of styles of communication in a reliable and flexible way (Draves, 1990). It can handle asynchronous message passing, RPC, byte streams, and other forms as well. Mach's interprocess communication mechanism is based on that of its ancestors, RIG and Accent. Due to this evolution, the mechanism used has been optimized for the local case (one node) rather than the remote case (distributed system).

We will first explain the single-node case in considerable detail, and then come back to how it has been extended for networking. It should be noted that in these terms, a multiprocessor is a single node, so communication between processes on different CPUs within the same multiprocessor uses the local case.

8.4.1. Ports

The basis of all communication in Mach is a kernel data structure called a **port**. A port is essentially a protected mailbox. When a thread in one process wants to communicate with a thread in another process, the sending thread

writes the message to the port and the receiving thread takes it out. Each port is protected to ensure that only authorized processes can send to it and receive from it.

Ports support unidirectional communication, like pipes in UNIX. A port that can be used to send a request from a client to a server cannot also be used to send the reply back from the server to the client. A second port is needed for the reply.

Ports support reliable, sequenced, message streams. If a thread sends a message to a port, the system guarantees that it will be delivered. Messages are never lost due to errors, overflow, or other causes (at least if there are no crashes). Messages sent by a single thread are also guaranteed to be delivered in the order sent. If two threads write to the same port in an interleaved fashion, taking turns, the system does not provide any guarantee about message sequencing, since some buffering may take place in the kernel due to locking and other factors.

Unlike pipes, ports support message streams, not byte streams. Messages are never concatenated. If a thread writes five 100-byte messages to a port, the receiver will always see them as five distinct messages, never as a single 500-byte message. Of course, higher-level software can ignore the message boundaries if they are not important to it.

A port is shown in Fig. 8-13. When a port is created, 64 bytes of kernel storage space are allocated and maintained until the port is destroyed, either explicitly, or implicitly under certain conditions, for example, when all the processes that are using it have exited. The port contains the fields shown in Fig. 8-13 and a few others.

Messages are not actually stored in the port itself but in another kernel data structure, the **message queue**. The port contains a count of the number of messages currently present in the message queue and the maximum permitted. If the port belongs to a port set, a pointer to the port set data structure is present in the port. As we mentioned briefly above, a process can give other processes capabilities to use its ports. For various reasons, the kernel has to know how many capabilities of each type are outstanding, so the port stores the counts.

If certain errors occur when using the port, they are reported by sending messages to other ports whose capabilities are stored there. Threads can block when reading from a port, so a pointer to the list of blocked threads is included. It is also important to be able to find the capability for reading from the port (there can only be one), so that information is present too. If the port is a process port, the next field holds a pointer to the process it belongs to. If it is a thread port, the field holds a pointer to the kernel's data structure for the thread, and so on. A few miscellaneous fields not described here are also needed.

When a thread creates a port, it gets back an integer identifying the port, analogous to a file descriptor in UNIX. This integer is used in subsequent calls

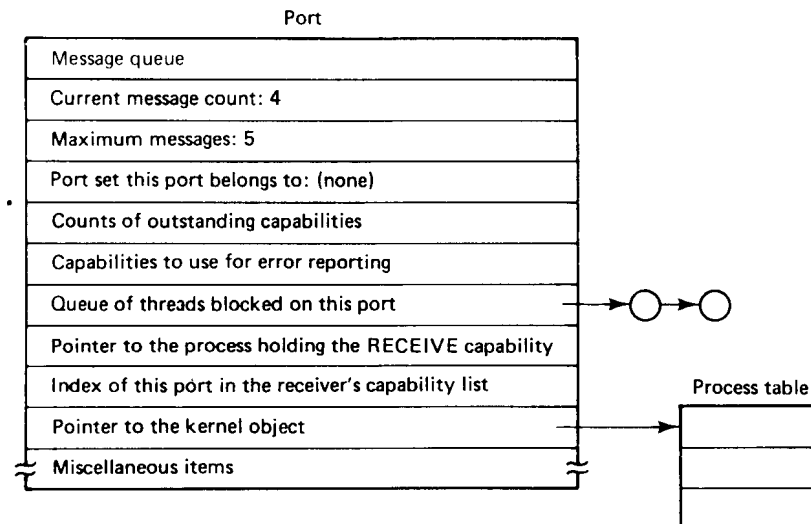


Fig. 8-13. A Mach port.

that send messages to the port or receive messages from it in order to identify which port is to be used. Ports are kept track of per process, not per thread, so if one thread creates a port and gets back the integer 3 to identify it, another thread in the same process will never get 3 to identify its new port. The kernel, in fact, does not even maintain a record of which thread created which port.

A thread can pass port access to another thread in a different process. Clearly, it cannot do so merely by putting the appropriate integer in a message, any more than a UNIX process can pass a file descriptor for standard output through a pipe by writing the integer 1 to the pipe. The exact mechanism used is protected by the kernel and will be discussed later. For the moment, it is sufficient to know that it can be done.

In Fig. 8-14 we see a situation in which two processes, *A* and *B*, each have access to the same port. *A* has just sent a message to the port, and *B* has just read the message. The header and body of the message are physically copied from *A* to the port and later from the port to *B*.

Ports may be grouped into **port sets** for convenience. A port may belong to at most one port set. It is possible to read from a port set (but not write to one). A server, for example, can use this mechanism to read from a large number of ports at the same time. The kernel returns one message from one of the ports in the set. No promises are made about which port will be selected. If all the ports are empty, the server is blocked. In this way a server can maintain a different port for each of the many objects that it supports, and get messages for any of

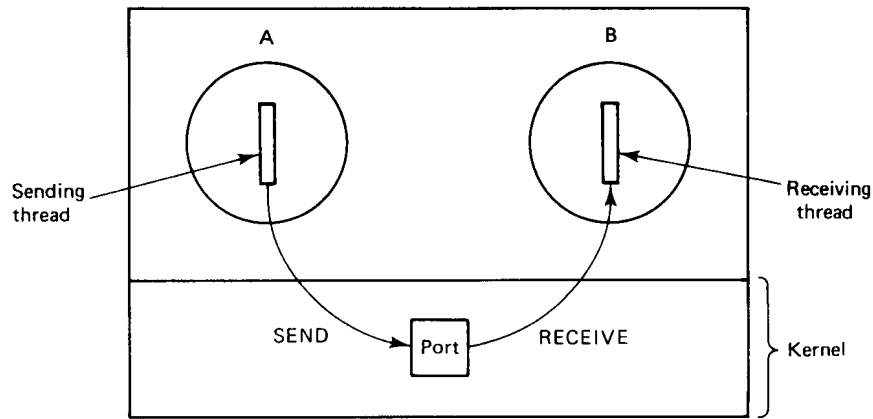


Fig. 8-14. Message passing goes via a port.

them without having to dedicate a thread to each one. The current implementation queues all the messages for the port set onto a single chain. In practice, the only difference between receiving from a port and receiving from a port set is that in the latter, the actual port sent to is identified to the receiver and in the former it is not.

Some ports are used in special ways. Every process has a special **process port** that it needs to communicate with the kernel. Most of the “system calls” associated with processes (see Fig. 8-3) are done by writing messages to this port. Similarly, each thread also has its own port for doing the “system calls” related to threads. Communication with I/O drivers also uses the port mechanism.

Capabilities

To a first approximation, for each process, the kernel maintains a table of all ports to which the process has access. This table is kept safely inside the kernel, where user processes cannot get at it. Processes refer to ports by their position in this table, that is, entry 1, entry 2, and so on. These table entries are effectively classical capabilities. We will refer to them as capabilities and will call the table containing the capabilities a **capability list**.

Each process has exactly one capability list. When a thread asks the kernel to create a port for it, the kernel does so and enters a capability for it in the capability list for the process to which the thread belongs. The calling thread and all the other threads in the same process have equal access to the capability. The

integer returned to the thread to identify the capability is usually an index into the capability list (but it can also be a large integer, such as a machine address). We will refer to this integer as a **capability name**, (or sometimes just a capability, where the context makes it clear that we mean the index and not the capability itself). It is always a 32-bit integer, never a string.

Each capability consists not only of a pointer to a port, but also a rights field telling what access the holder of the capability has to the port. (All the threads in a process are equally considered holders of the process' capabilities.) Three rights exist: RECEIVE, SEND, and SEND-ONCE. The RECEIVE right gives the holder the ability to read messages from the port. Earlier we mentioned that communication in Mach is unidirectional. What this really means is that at any instant only one process may have the RECEIVE right for a port. A capability with a RECEIVE right may be transferred to another process, but doing so causes it to be removed from the sender's capability list. Thus for each port there is a single potential receiver.

A capability with the SEND right allows the holder to send messages to the specified port. Many processes may hold capabilities to send to a port. This situation is roughly analogous to the banking system in most countries: anyone who knows a bank account number can deposit money to that account, but only the owner can make withdrawals.

The SEND-ONCE right also allows a message to be sent, but only one time. After the send is done, the kernel destroys the capability. This mechanism is used for request-reply protocols. For example, a client wants something from a server, so it creates a port for the reply message. It then sends the server a request message containing a (protected) capability for the reply port with the SEND-ONCE right. After the server sends the reply, the capability is deallocated from its capability list and the name is made available for a new capability in the future.

Capability names have meaning only within a single process. It is possible for two processes to have access to the same port but use different names for it, just as two UNIX processes may have access to the same open file but use different file descriptors to read it. In Fig. 8-15 both processes have a capability to send to port *Y*, but in *A* it is capability 3 and in *B* it is capability 4.

A capability list is tied to a specific process. When that process exits or is killed, its capability list is removed. Ports for which it holds a capability with the RECEIVE right are no longer usable and are therefore also destroyed, even if they contain undelivered (and now undeliverable) messages.

If different threads in a process acquire the same capability multiple times, only one entry is made in the capability list. To keep track of how many times each is present, the kernel maintains a reference count for each port. When a capability is deleted, the reference count is decremented. Only when it gets to zero is the capability actually removed from the capability list. This mechanism

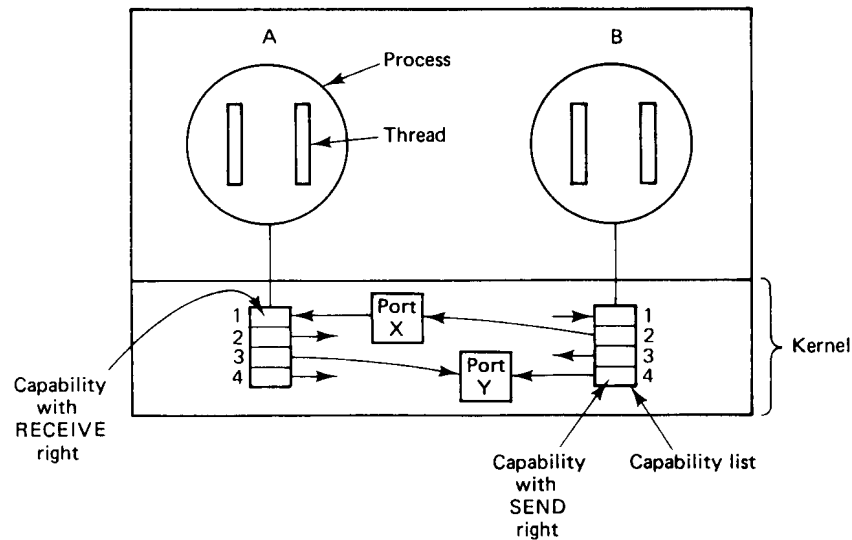


Fig. 8-15. Capability lists.

is important because different threads may acquire and release capabilities without each other's knowledge, for example, the UNIX emulation library and the program being run.

Each capability list entry is one of the following four items:

1. A capability for a port.
2. A capability for a port set.
3. A null entry.
4. A code indicating that the port that was there is now dead.

The first possibility has already been explained in some detail. The second allows a thread to read from a set of ports without even being aware that the capability name is backed up by a set rather than by a single port. The third is a place holder that indicates that the corresponding entry is not currently in use. If an entry is allocated for a port that is later destroyed, the capability is replaced by a null entry to mark it as unused.

Finally, the fourth option marks ports that no longer exist but for which capabilities with SEND rights still exist. When a port is deleted, for example, because the process holding the RECEIVE capability for it has exited, the kernel tracks down all the SEND capabilities and marks them as dead. Attempts to send to null and dead capabilities fail with an appropriate error code. When all

the SEND capabilities for a port are gone, for whatever reasons, the kernel (optionally) sends a message notifying the receiver that there are no senders left and no messages will be forthcoming.

Primitives for Managing Ports

Mach provides about 20 calls for managing ports. All of these are invoked by sending a message to a process port. A sampling of the most important ones is given in Fig. 8-16.

Call	Description
Allocate	Create a port and insert its capability in the capability list
Destroy	Destroy a port and remove its capability from the list
Deallocate	Remove a capability from the capability list
Extract_right	Extract the n-th capability from another process
Insert_right	Insert a capability in another process' capability list
Move_member	Move a capability into a capability set
Set_qlimit	Set the number of messages a port can hold

Fig. 8-16. Selected port management calls in Mach.

The first one, *allocate*, creates a new port and enters its capability into the caller's capability list. The capability is for reading from the port. A capability name is returned so that the port can be used.

The next two undo the work of the first. *Destroy* removes a capability. If it is a RECEIVE capability, the port is destroyed and all other capabilities for it in all processes are marked as dead. *Deallocate* decrements the reference count associated with a capability. If it is zero, the capability is removed but the port remains intact. *Deallocate* can only be used to remove SEND or SEND-ONCE capabilities or dead capabilities.

Extract_right allows a thread to select out a capability from another process' capability list and insert the capability in its own list. Of course, the calling thread needs access to the process port controlling the other process (e.g., its own child). *Insert_right* goes the other way. It allows a process to take one of its own capabilities and add it to (for example) a child's capability list.

The *move_member* call is used for managing port sets. It can add a port to a port set or remove one. Finally, *set_qlimit* determines the number of messages a

port can hold. When a port is created, the default is five messages, but with this call that number can be increased or decreased. The messages can be of any size since they are not physically stored in the port itself.

8.4.2. Sending and Receiving Messages

The purpose of having ports is to send messages to them. In this section we will look at how messages are sent, how they are received, and what they contain. Mach has a single system call for sending and receiving messages. The call is wrapped in a library procedure called *mach_msg*. It has seven parameters and a large number of options. To give an idea of its complexity, there are 35 different error messages that it can return. Below we will give a simplified sketch of some of its possibilities. Fortunately, it is used primarily in procedures generated by the stub compiler, rather than being written by hand.

The *mach_msg* call is used for both sending and receiving. It can send a message to a port and then return control to the caller immediately, at which time the caller can modify the message buffer without affecting the data sent. It can also try to receive a message from a port, blocking if the port is empty, or giving up after a certain interval. Finally, it can combine these two operations, first sending a message and then blocking until a reply comes back. In the latter mode, *mach_msg* can be used for RPC.

A typical call to *mach_msg* looks like this:

```
mach_msg(&hdr,options,send_size,rcv_size,rcv_port,timeout,notify_port);
```

The first parameter, *hdr*, is a pointer to the message to be sent or to the place where the incoming message is put, or both. The message begins with a fixed header and is followed directly by the message body. This layout is shown in Fig. 8-17. We will explain the details of the message format later, but for the moment just note that the header contains a capability name for the destination port. This information is needed so that the kernel can tell where to send the message. When doing a pure RECEIVE, the header is not filled in, since it will be overwritten entirely by the incoming message.

The second parameter of the *mach_msg* call, *options*, contains a bit specifying that a message is to be sent, and another one specifying that a message is to be received. If both are on, an RPC is done. Another bit enables a timeout, given by the *timeout* parameter, in milliseconds. If the requested operation cannot be performed within the timeout interval, the call returns with an error code. If the SEND portion of an RPC times out (e.g., due to the destination port being full too long), the RECEIVE is not even attempted.

Other bits in *options* allow a SEND that cannot complete immediately to return control anyway, with a status report being sent to *notify_port* later. All kinds of errors can occur here if the capability for *notify_port* is unsuitable or

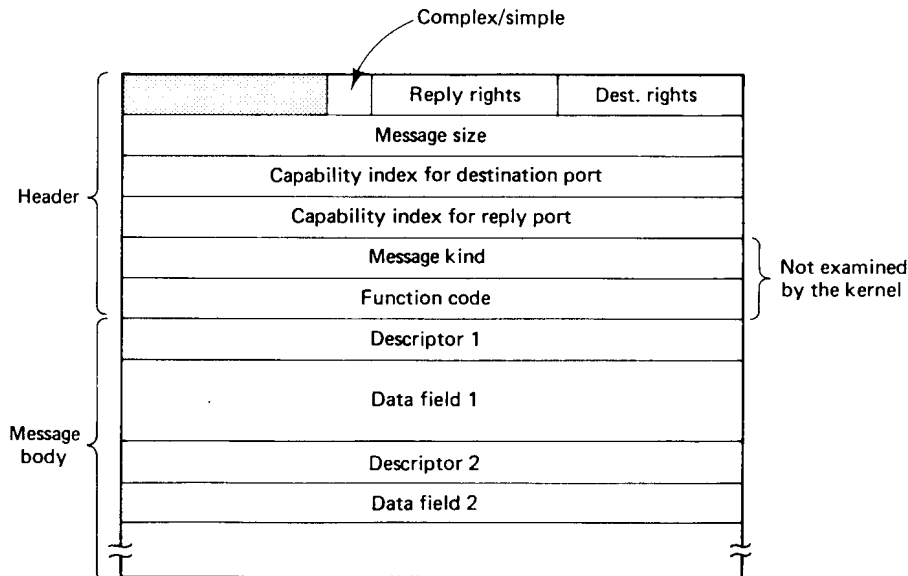


Fig. 8-17. The Mach message format.

changed before the notification can occur. It is even possible for the call to ruin *notify_port* itself (calls can have complex side effects, as we will see later).

The *mach_msg* call can be aborted part-way through by a software interrupt. Another *options* bit tells whether to give up or try again.

The *send_size* and *rcv_size* parameters tell how large the outgoing message is and how many bytes are available for storing the incoming message, respectively. *Rcv_port* is used for receiving messages. It is the capability name of the port or port set being listened to.

Now let us turn to the message format of Fig. 8-17. The first word contains a bit telling whether the message is simple or complex. The difference is that simple messages cannot carry capabilities or protected pointers, whereas complex ones can. Simple messages require less work on the part of the kernel and are therefore more efficient. Both message types have a system-defined structure, described below. The *message size* field tells how big the combined header plus body is. This information is needed both for transmission and by the receiver.

Next come two capability names (i.e., indices into the sender's capability list). The first specifies the destination port; the second can give a reply port. In client-server RPC, for example, the destination field designates the server and the reply field tells the server which port to send the response to.

The last two header fields are not used by the kernel. Higher levels of

software can use them as desired. By convention, they are used to specify the kind of message and give a function code or operation code (e.g., to a server, is this request for reading or for writing?). This usage is subject to change in the future.

When a message is sent and successfully received, it is copied into the destination's address space. It can happen, however, that the destination port is already full. What happens then depends on the various options and the rights associated with the destination port. One possibility is that the sender is blocked and simply waits until space becomes available in the port. Another is that the sender times out. In some cases, it can exceed the port limit and send anyway.

A few issues concerning receiving messages are worth mentioning. For one, if an incoming message is larger than the buffer, what should be done with it? Two options are provided: throw it away or have the *mach_msg* call fail but return the size, thus allowing the caller to try again with a bigger size.

If multiple threads are blocked trying to read from the same port and a message arrives, one of them is chosen by the system to get the message. The rest remain blocked. If the port being read from is actually a port set, it is possible for the composition of the set to change while one or more threads are blocked on it. This is probably not the place to go into all the details, but suffice it to say that there are precise rules governing this and similar situations.

Message Formats

A message body can be either simple or complex, controlled by a header bit, as mentioned above. Complex messages are structured as shown in Fig. 8-17. A complex message body consists of a sequence of (descriptor, data field) pairs. Each descriptor tells what is in the data field immediately following it. Descriptors come in two formats, differing only in how many bits each of the fields contains. The normal descriptor format is illustrated in Fig. 8-18. It specifies the type of the item that follows, how large an item is, and how many of them there are (a data field may contain multiple items of the same type). The available types include raw bits and bytes, integers of various sizes, unstructured machine words, collections of Booleans, floating-point numbers, strings, and capabilities. Armed with this information, the system can attempt to do conversions between machines when the source and destination machines have different internal representations. This conversion is not done by the kernel but by the network message server (described below). It is also done for internode transport even for simple messages (also by the network message server).

One of the more interesting items that can be contained in a data field is a capability. Using complex messages it is possible to copy or transfer a capability from one process to another. Because capabilities are protected kernel objects in Mach, a protected mechanism is needed to move them about.

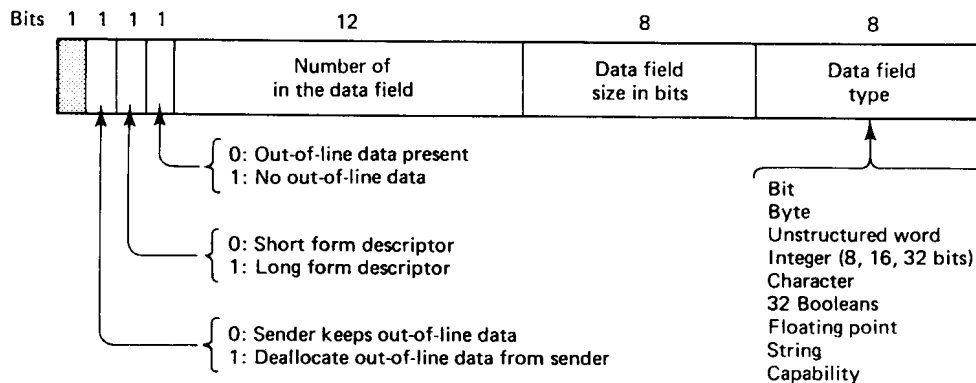


Fig. 8-18. A complex message field descriptor.

This mechanism is as follows. A descriptor can specify that the word directly after it in the message contains the name for one of the sender's capabilities, and that this capability is to be passed to the receiving process and inserted in the receiver's capability list. The descriptor also specifies whether the capability is to be copied (the original is not touched) or moved (the original is deleted).

Furthermore, certain values of the *Data field type* ask the kernel to modify the capability's rights while doing the copy or move. A RECEIVE capability, for example, can be mutated into a SEND or SEND-ONCE capability, so that the receiver will have the power to send a reply to a port for which the sender has only a RECEIVE capability. In fact, the normal way to establish communication between two processes is to have one of them create a port and then send the port's RECEIVE capability to the other one, turning it into a SEND capability in flight.

To see how capability transport works, consider Fig. 8-19(a). Here we see two processes, *A* and *B*, with 3 capabilities and 1 capability, respectively. All are RECEIVE capabilities. Numbering starts at 1 since entry 0 is the null port. One of the threads in *A* is sending a message to *B* containing capability 3.

When the message arrives, the kernel inspects the header and sees that it is a complex message. It then begins processing the descriptors in the message body, one by one. In this example there is only one descriptor, for a capability, with instructions to turn it into a SEND (or maybe SEND-ONCE) capability. The kernel allocates a free slot in the receiver's capability list, slot 2 in this example, and modifies the message so that the word following the descriptor is now 2 instead of 3. When the receiver gets the message, it sees that it has a new capability, with name (index) 2. It can use this capability immediately (e.g., for sending a reply message).

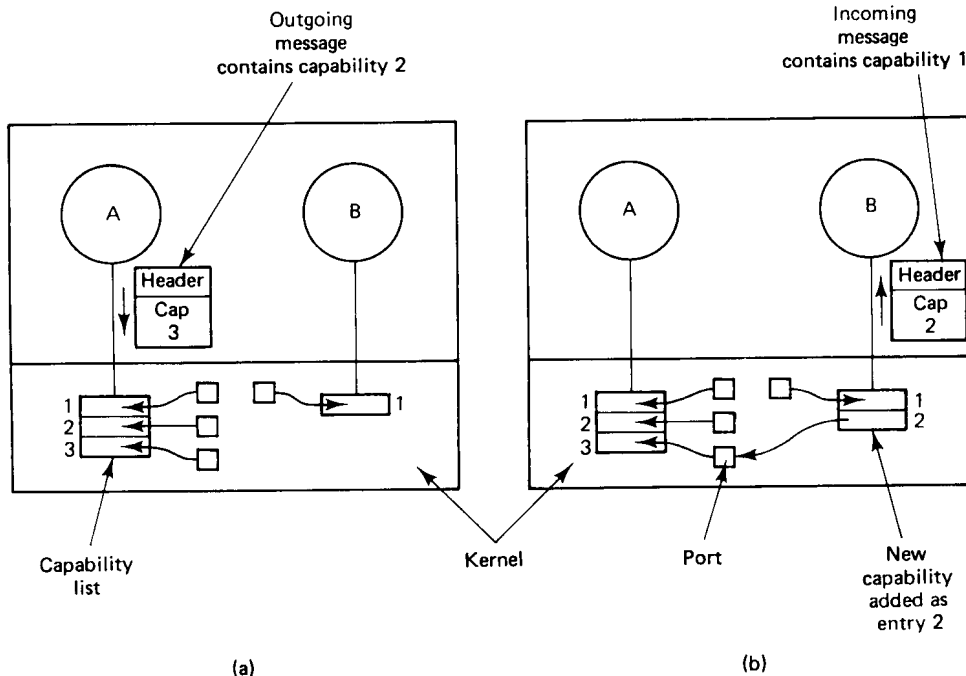


Fig. 8-19. (a) Situation just before the capability is sent. (b) Situation after it has arrived.

There is one last aspect of Fig. 8-18 that we have not yet discussed: **out-of-line data**. Mach provides a way to transfer bulk data from a sender to a receiver without doing any copying (on a single machine or multiprocessor). If the out-of-line data bit is set in the descriptor, the word following the descriptor contains an address, and the size and number fields of the descriptor give a 20-bit byte count. Together these specify a region of the sender's virtual address space. For larger regions, the long form of the descriptor is used.

When the message arrives at the receiver, the kernel chooses an unallocated piece of virtual address space the same size as the out-of-line data, and maps the sender's pages into the receiver's address space, marking them copy-on-write. The address word following the descriptor is changed to reflect the address at which the region is located in the receiver's address space. This mechanism provides a way to move blocks of data at extremely high speed, because no copying is required except for the message header and the two-word body (the descriptor and the address). Depending on a bit in the descriptor, the region is either removed from the sender's address space or kept there.

Although this method is highly efficient for copies between processes on a single machine (or between CPUs in a multiprocessor), it is not as useful for

communication over a network because the pages must be copied if they are used, even if they are only read. Thus the ability to transmit data logically without moving physically them is lost. Copy-on-write also requires that messages be aligned on page boundaries and be an integral number of pages in length for best results. Fractional pages allow the receiver to see data before or after the out-of-line data that it should not see.

8.4.3. The Network Message Server

Everything we have said so far about communication in Mach is limited to communication within a single node, either one CPU or a multiprocessor node. Communication over the network is handled by user-level servers called **network message servers**, which are vaguely analogous to the external memory managers we studied earlier. Every machine in a Mach distributed system runs a network message server. The network message servers work together to handle intermachine messages, trying to simulate intramachine messages as best they can.

A network message server is a multithreaded process that performs a variety of functions. These include interfacing with local threads, forwarding messages over the network, translating data types from one machine's representation to another's, managing capabilities in a secure way, doing remote notification, providing a simple network-wide name lookup service, and handling authentication of other network message servers. Network message servers can speak a variety of protocols, depending on the networks to which they are attached.

The basic method by which messages are sent over the network is illustrated in Fig. 8-20. Here we have a client on machine *A* and a server on machine *B*. Before the client can contact the server, a port must be created on *A* to function as a proxy for the server. The network message server has the **RECEIVE** capability for this port. A thread inside it is constantly listening to this port (and other remote ports, which together form a port set). This port is shown as the small box in *A*'s kernel.

Message transport from the client to the server requires five steps, numbered 1 to 5 in Fig. 8-20. First, the client sends a message to the server's proxy port. Second, the network message server gets this message. Since this message is strictly local, out-of-line data may be sent to it and copy-on-write works in the usual way. Third, the network message server looks up the local port, 4 in this example, in a table that maps proxy ports onto **network ports**. Once the network port is known, the network message server looks up its location in other tables. It then constructs a network message containing the local message, plus any out-of-line data and sends it over the LAN to the network message server on the server's machine. In some cases, traffic between the network message servers has to be encrypted for security. The transport module takes care of

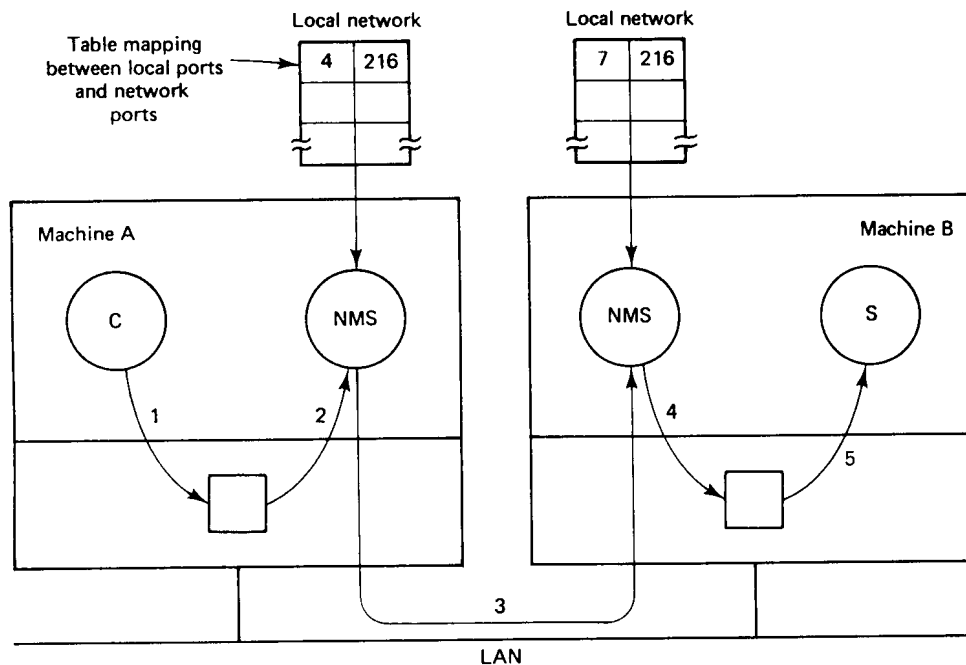


Fig. 8-20. Intermachine communication in Mach proceeds in five steps.

breaking the message into packets and encapsulating them in the appropriate protocol wrappers.

When the remote network message server gets the message, it looks up the network port number contained in it and maps it onto a local port number. In step 4, it writes the message to the local port just looked up. Finally, the server reads the message from the local port and carries out the request. The reply follows the same path in the reverse direction.

Complex messages require a bit more work. For ordinary data fields, the network message server on the server's machine must perform conversion, if necessary, for example, taking account of different byte ordering on the two machines. Capabilities must also be processed. When a capability is sent over the network, it must be assigned a network port number, and both the source and destination network message servers must make entries for it in their mapping tables. If these machines do not trust each other, elaborate authentication procedures will be necessary to convince each machine of the other's true identity.

Although the idea of relaying messages from one machine to another via a user-level server offers some flexibility, a substantial price is paid in performance as compared to a pure kernel implementation, which most other distributed systems use. To solve this problem, a new version of the network

communication package is being developed (the NORMA code), which runs inside the kernel and achieves faster communication. It will eventually replace the network message server.

8.5. UNIX EMULATION IN MACH

Mach has various servers that run on top of it. Probably the most important one is a program that contains a large amount of Berkeley UNIX (e.g., essentially the entire file system code) inside itself. This server is the main UNIX emulator (Golub et al., 1990). This design is a legacy of Mach's history as a modified version of Berkeley UNIX.

The implementation of UNIX emulation on Mach consists of two pieces, the UNIX server and a system call emulation library, as shown in Fig. 8-21. When the system starts up, the UNIX server instructs the kernel to catch all system call traps and vector them to addresses inside the emulation library of the UNIX process making the system call. From that moment on, any system call made by a UNIX process will result in control passing temporarily to the kernel and immediately thereafter passing to its emulation library. At the moment control is given to the emulation library, all the machine registers have the values they had at the time of the trap. This method of bouncing off the kernel back into user space is sometimes called the **trampoline mechanism**.

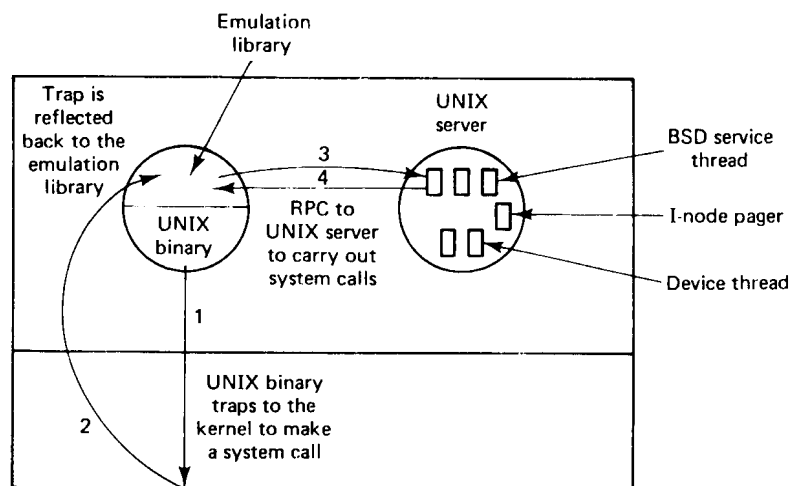


Fig. 8-21. UNIX emulation in Mach uses the trampoline mechanism.

Once the emulation library gets control, it examines the registers to determine which system call was invoked. It then makes an RPC to another process,

the UNIX server, to do the work. When it is finished, the user program is given control again. This transfer of control need not go through the kernel.

When the *init* process forks off children, they automatically inherit both the emulation library and the trampoline mechanism, so they, too, can make UNIX system calls. The EXEC system call has been changed so that it does not replace the emulation library but just the UNIX program part of the address space.

The UNIX server is implemented as a collection of C threads. Although some threads handle timers, networking, and other I/O devices, most threads handle BSD system calls, carrying out requests on behalf of the emulators inside the UNIX processes. The emulation library communicates with these threads using the usual Mach interprocess communication.

When a message comes in to the UNIX server, an idle thread accepts it, determines which process it came from, extracts the system call number and parameters from it, carries it out, and finally, sends back the reply. Most messages correspond exactly to one BSD system call.

One set of system calls that work differently are the file I/O calls. They *could* have been implemented like this, but for performance reasons, a different approach was taken. When a file is opened, it is mapped directly into the caller's address space, so the emulation library can get at it directly, without having to do an RPC to the UNIX server. To satisfy a READ system call, for example, the emulation library locates the bytes to be read in the mapped file, locates the user buffer, and copies from the former to the latter as fast as it can.

Page faults will occur during the copy loop if the file's pages are not in memory. Each fault will cause Mach to send a message to the external memory manager backing up the mapped UNIX file. This memory manager is a thread inside the UNIX server called the **i-node pager**. It gets the file page from the disk and arranges for it to be mapped into the application program's address space. It also synchronizes operations on files that are open by several UNIX processes simultaneously.

Although this method of running UNIX programs looks cumbersome, measurements have shown that it compares favorably with traditional monolithic kernel implementations (Golub et al., 1990). Future work will focus on splitting the UNIX server into multiple servers with more specific functions. Eventually, the single UNIX server may be eliminated, although this depends on how the work with multiple servers develops during the course of time.

8.6. SUMMARY

Mach is a microkernel-based operating system. It was designed as a base for building new operating systems and emulating existing ones. It also provides a flexible way to extend UNIX to multiprocessors and distributed systems.

Mach is based on the concepts of processes, threads, ports, and messages. A Mach process is an address space and a collection of threads that run in it. The active entities are the threads. The process is merely a container for them. Each process and thread has a port to which it can write to have kernel calls carried out, eliminating the need for direct system calls.

Mach has an elaborate virtual memory system, featuring memory objects that can be mapped and unmapped into address spaces, and backed up by external, user-level memory managers. Files can be made directly readable and writable in this way, for example. Memory objects can be shared in various ways, including copy-on-write. Inheritance attributes determine which parts of a process' address space will be passed to its children.

Communication in Mach is based on ports, which are kernel objects that hold messages. All messages are directed to ports. Ports are accessed using capabilities, which are stored inside the kernel and referred to by 32-bit integers that are usually indices into capability lists. Ports can be passed from one process to another by including them in complex messages.

BSD UNIX emulation is done by an emulation library that lives in the address space of each UNIX process. Its job is to catch system calls reflected back to it by the kernel, and pass them on to the UNIX server to have them carried out. A few calls are handled locally, within the process' address space. Other UNIX emulators are also being developed.

Amoeba and Mach have many aspects in common, but also various differences. Both have processes and threads and are based on message passing. Amoeba has reliable broadcasting as a primitive, which Mach does not, but Mach has demand paging, which Amoeba does not. In general, Amoeba is more oriented toward making a collection of distributed machines act like a single computer, whereas Mach is more oriented toward making efficient use of multi-processors. Both are undergoing constant development and will no doubt change as time goes on.

PROBLEMS

1. Name one difference between a process with two threads and two processes each with one thread that share the same address space, that is, the same set of pages.
2. What happens if you *join* on yourself?
3. A Mach thread creates two new threads as its children, *A* and *B*. Thread *A* does a *detach* call; *B* does not. Both threads exit and the parent does a *join*. What happens?

4. The global run queues of Fig. 8-6 must be locked before being searched. Do the local run queues (not shown in the figure) also have to be locked before being searched? Why or why not?
5. Each of the global run queues has a single mutex for locking it. Suppose that a particular multiprocessor has a global clock that causes clock interrupts on all the CPUs simultaneously. What implications does this have for the Mach scheduler?
6. Mach supports the concept of a processor set. On what class of machines does this concept make the most sense? What is it used for?
7. Mach supports three inheritance attributes for regions of virtual address space. Which ones are needed to make UNIX FORK work correctly?
8. A small process has all its pages in memory. There is enough free memory available for ten more copies of the process. It forks off a child. Is it possible for the child to get a page or protection fault?
9. Why do you think there is a call to copy a region of virtual memory (see Fig. 8-8)? After all, any thread can just copy it by sitting in a tight copy loop.
10. Why is the page replacement algorithm run in the kernel instead of in an external memory manager?
11. Give an example when it is desirable for a thread to deallocate an object in its virtual address space.
12. Can two processes simultaneously have RECEIVE capabilities for the same port? How about SEND capabilities?
13. Does a process know that a port it is reading from is actually a port set? Does it matter?
14. Mach supports two types of messages: simple and complex. Are the complex messages actually required, or is this merely an optimization?
15. Now answer the previous question about SEND-ONCE capabilities and out-of-line messages. Are either of these essential to the correct functioning of Mach?
16. In Fig. 8-15 the same port has a different name in different processes. What problems might this cause?
17. Mach has a system call that allows a process to request that non-Mach traps be given to a special handler, rather than causing the process to be killed. What is this system call good for?

9

Case Study 3: Chorus

Our third example of a modern, microkernel-based operating system is Chorus. The structure of this chapter is similar to that of the previous two: first a brief history, then an overview of the microkernel, followed by a more detailed look at process management, memory management, and communication. After that, we will study how Chorus tackles UNIX emulation. Next comes a section on distributed object-oriented programming in Chorus. We will conclude with a short comparison of Amoeba, Mach, and Chorus. More information about Chorus can be found in (Abrossimov et al., 1989, 1992; Armand and Dean, 1992; Batlivala, et al., 1992; Bricker et al., 1991; Gien and Grob, 1992; and Rozier et al., 1988).

9.1. INTRODUCTION TO CHORUS

In this section we will summarize how Chorus has evolved over the years, discuss its goals briefly, and then give a technical introduction to its microkernel and two of its subsystems. In subsequent sections we will describe the kernel and subsystems in more detail. The Chorus documentation uses a somewhat nonstandard terminology. In this chapter we will use the standard names but give the Chorus terms in parentheses.

9.1.1. History of Chorus

Chorus started out at the French research institute INRIA in 1980, as a research project in distributed systems. It has since gone through four versions, numbered from 0 through 3. The idea behind Version 0 was to model distributed applications as a collection of **actors**, essentially structured processes, each of which alternated between performing an atomic transaction and executing a communication step. In effect, each actor was a macroscopic finite-state automaton. Each machine in the system ran the same kernel, which managed the actors, communication, files, and I/O devices. Version 0 was written in interpreted UCSD Pascal and ran on a collection of 8086s connected by a ring network. It was operational by mid-1982.

Version 1, which lasted from 1982 to 1984, focused on multiprocessor research. It was written for the French SM90 multiprocessor, which consisted of eight Motorola 68020 CPUs on a common bus. One of the CPUs ran UNIX; the other seven ran Chorus and used the UNIX CPU for system services and I/O. Multiple SM90s were connected by an Ethernet. The software was similar to Version 0, with the addition of structured messages and some support for fault tolerance. Version 1 was written in compiled, rather than interpreted, Pascal and was distributed to about a dozen universities and companies for experimental use.

Version 2 (1984-1986) was a major rewrite of the system, in C. It was designed to be system call compatible with UNIX at the source code level, meaning that it was possible to recompile existing UNIX programs on Chorus and have them run on it. The Version 2 kernel was completely redesigned, moving as much functionality as possible from it to user code, and turning the kernel into what is now regarded as a microkernel. The UNIX emulation was done by several processes, for handling process management, file management, and device management, respectively. Support was added for distributed applications, including remote execution and protocols for distributed naming and location.

Version 3 was started in 1987. This version marked the transition from a research system to a commercial product, as the Chorus designers left INRIA and formed a company, Chorus Systèmes, to further develop and market Chorus. Numerous technical changes were made in Version 3, including further refinement of the microkernel and its relation to the rest of the system. The last vestiges of the actor model, with its atomic transactions, disappeared, and RPC (Remote Procedure Call) was introduced as the usual communication model. Kernel mode processes also appeared.

To make Chorus a viable commercial product, the ability to emulate UNIX was beefed up. Binary compatibility was added, so existing UNIX programs could be run without being recompiled. Part of the UNIX emulation, which had been in the microkernel, was moved to the emulation subsystem, which was

simultaneously made more modular. Exception handling was changed to be able to handle UNIX signals correctly.

Its performance was improved. Also, the system was partially rewritten in C++. Furthermore, it was made more portable and implemented on a number of different architectures. Version 3 also borrowed many ideas from other distributed system microkernels, notably the interprocess communication system, virtual memory design, and external pagers from Mach, and the use of sparse capabilities for global naming and protection from Amoeba.

9.1.2. Goals of Chorus

The goals of the Chorus project have evolved along with the system itself. Initially, it was pure academic research, designed to explore new ideas in distributed computing based on the actor model. As time went on, it became more commercial, and the emphasis shifted. The current goals can be roughly summarized as follows:

1. High-performance UNIX emulation.
2. Use on distributed systems.
3. Real-time applications.
4. Integrating object-oriented programming into Chorus.

As a commercial system, much work focuses on tracking evolving UNIX standards, porting the system to new CPU chips, and improving performance. The company wants Chorus to be seen as an alternative to AT&T UNIX, re-engineered, easier to maintain, and oriented to future user requirements.

A second major theme is the need for distribution. Chorus is intended to allow UNIX programs to run on a collection of machines connected by a network. To support distributed applications, various extensions have been added to the programming model. Some of these, such as message-based communication, fit easily in the existing model. Others, such as the introduction of threads, required a rethinking of existing features, such as UNIX signal handling.

A third direction is the introduction of support for real-time applications. The approach taken is to allow real-time programs to run (partly) in kernel mode and have direct access to the microkernel, without any software in the way. User control over interrupts and the scheduling algorithm are also important here.

Finally, another goal is the introduction of object-oriented programming into Chorus in a clean way, without disturbing existing subsystems and applications. How this is being done will be described in detail later in this chapter.

9.1.3. System Structure

Chorus is structured in layers, as illustrated in Fig. 9-1. At the bottom is the microkernel (called the **nucleus** in the Chorus documentation). It provides minimal management of names, processes, threads, memory, and communication. These services are accessed by calls to the microkernel. Over 100 calls exist. Processes in higher layers provide the rest of the operating system. Every machine in a distributed system based on Chorus runs an identical copy of the Chorus microkernel.

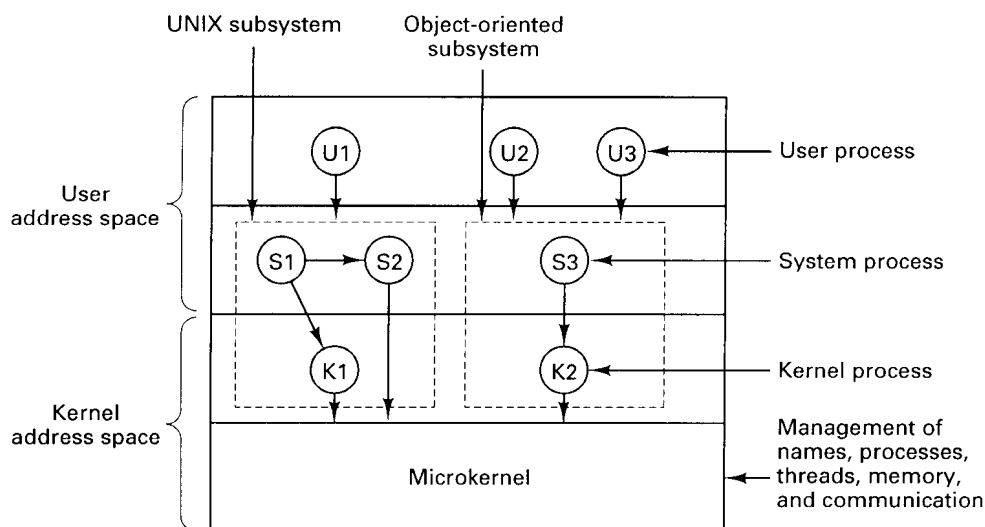


Fig. 9-1. Chorus is structured in layers, with a microkernel, subsystems, and user processes.

On top of the microkernel, but also operating in kernel mode, are the **kernel processes**. These processes can be dynamically loaded and removed during system execution and provide a way to extend the functionality of the microkernel without permanently increasing its size and complexity. Since these processes share the kernel space with the microkernel and with each other, they must be relocated after being loaded. They can invoke the microkernel to obtain services, and can call one another as well.

For example, interrupt handlers are written as kernel processes. On a machine with a disk drive, at system initialization time, the disk interrupt handler process will be loaded. When disk interrupts occur, they will be handled by this process. On diskless workstations, the disk interrupt handler is not needed and will not be loaded. The ability to dynamically load and unload kernel processes makes it possible to configure the system software to match the hardware, without having to recompile or relink the microkernel.

The next layer contains the **system processes**. These run in user mode, but can send messages to kernel processes (and each other) and can make calls to the microkernel, as shown by the arrows in Fig. 9-1. A collection of kernel and system processes can work together to form a **subsystem**. In Fig. 9-1, processes *S1*, *S2*, and *K1* form one subsystem and processes *S3* and *K2* form a second one. A subsystem presents a well-defined interface to its users, such as the UNIX system call interface. One process in each subsystem is the manager and controls the operation of the subsystem.

On top of the subsystems are the **user processes**. For example, system calls made by a user process *U1* might be caught by *K1* and passed on to *S1* or *S2* for processing. These, in turn, could use microkernel services, where appropriate. Subsystems make it possible to build new (or old) operating systems on top of the microkernel in a modular way, and to allow multiple operating system interfaces to exist on the same machine at the same time.

The microkernel knows which subsystem (if any) each user process is using, and ensures that it is restricted to making the system calls offered by that subsystem. Direct calls from user processes to the microkernel are not permitted, except for those calls that the subsystem defines as legal. Real-time processes can run as system processes, rather than as user processes, and thus make full use of the microkernel without intervention or overhead.

9.1.4. Kernel Abstractions

The kernel (by which we mean the microkernel) provides and manages six key abstractions that together form the basis for Chorus. These concepts are processes, threads, regions, messages, ports, port groups, and unique identifiers. They are illustrated in Fig. 9-2. Chorus **processes** (still called **actors** in the Chorus documentation) are essentially the same as processes in other operating systems. They are containers that encapsulate resources. A process owns certain resources, and when the process disappears, so do its resources.

Within a process, one or more **threads** can exist. Each thread is similar to a process in that it has its own stack, stack pointer, program counter, and registers. However, all threads in a process share the same address space and other process-wide resources. In principle, the threads of a process are independent of one another. On a multiprocessor, several threads may be running at the same time, on different CPUs. All three kinds of processes can have multiple threads.

Each process has an address space, normally going from 0 to some maximum address, such as $2^{32} - 1$. All the threads in a process have access to this address space. A consecutive range of addresses is called a **region**. Each region is associated with some piece of data, such as a program or a file. In systems that support virtual memory and paging, regions may be paged. Regions play a major role in memory management in Chorus.

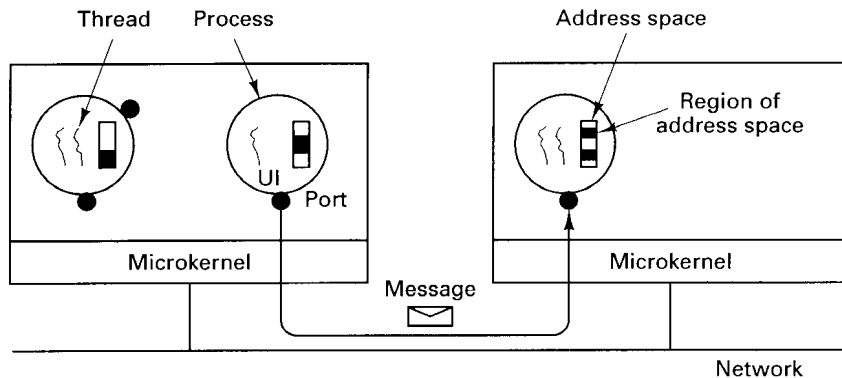


Fig. 9-2. Processes, threads, regions, messages, and, ports are identified by UIs.

Threads in different processes (potentially on different machines) communicate by passing **messages**. Messages can have a fixed part and a variable-sized body, both of which are optional. The body is untyped and may contain whatever information the sender puts into it. A message is addressed not to a thread, but to an intermediate structure called a **port**. A port is a buffer for incoming messages and holds those messages received by a process but not yet read. Like a thread, region, or other resource, at any instant, each port belongs to a single process. Only that process can read its messages. Ports can be put together to form port groups. These groups will be discussed below.

The last kernel abstraction relates to naming. Most kernel resources (e.g., processes and ports) are named by a 64-bit **unique identifier** or **UI**. Once a UI has been assigned to a resource, it is guaranteed never to be reused for another resource, not even on a different machine a year later. This uniqueness is guaranteed by encoding in each UI the site (machine or multiprocessor) where the UI was created plus an epoch number and a counter valid in that epoch. The epoch number is incremented each time the system is rebooted.

UIs are just binary numbers and are themselves not protected. Processes can send UIs to other processes in messages or store them in files. When a UI is transferred across the network and the receiver tries to access the corresponding object, the location information in the UI is used as a hint for where the object might be.

Because UIs are long and expensive to use, **local identifiers** or **LIs** are used within a single process to identify resources, similar to the use of small integers as file descriptors to identify open files in UNIX.

The kernel abstractions are not the only ones used by Chorus. Three other abstractions that are jointly managed by the kernel and subsystems also are important. These are capabilities, protection identifiers, and segments. A